Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Detectar elementos en la web (Localizadores)
- Creación de scripts de pruebas para certificar la web
- Ejecución del script automatizado
- Integrar JUNIT con Selenium WebDriver
- Revisión de los resultados.
- Características avanzadas de script de pruebas.

idat

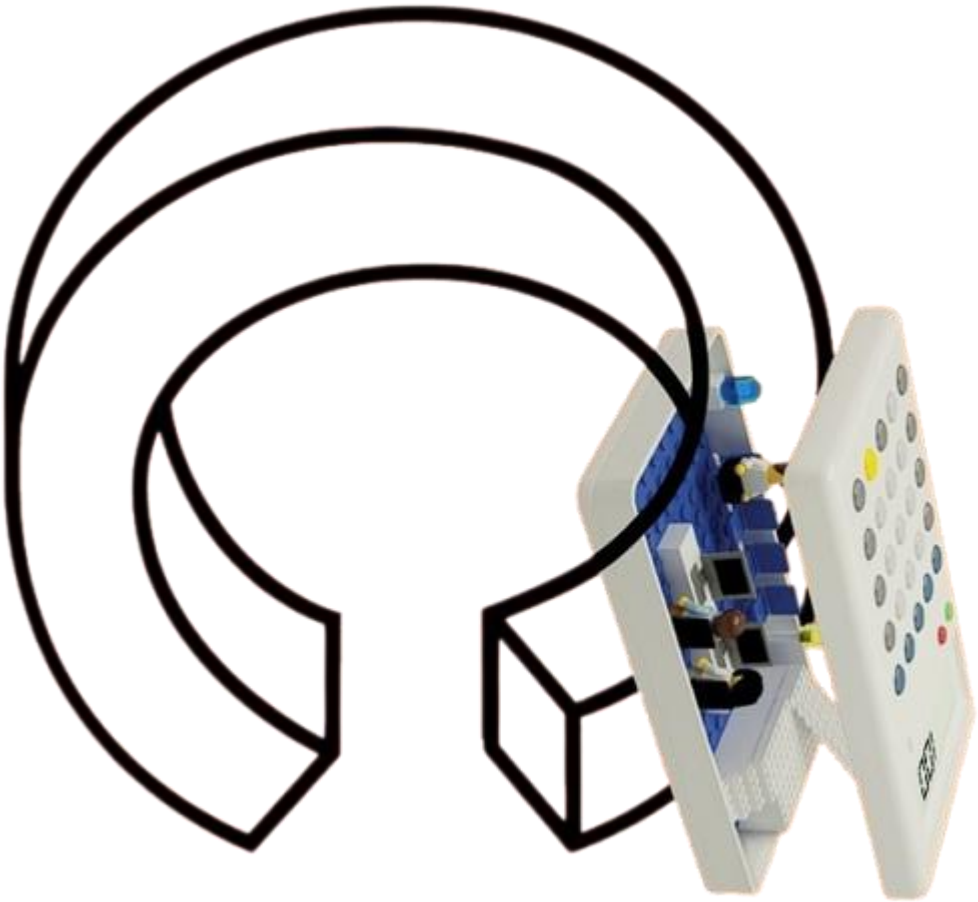
Conversemos

http://www.menti.com

idat

Introducción a pruebas agiles – Parte 1

Semana 12



idat

Logro de aprendizaje

- SCRUM
- Ciclo de vida SCRUM
- Roles y ceremonias
- Historia de usuarios
- Creación de casos de prueba basados en historias de usuario.
- Que es Cucumber y gherkin
- Configuración del entorno de pruebas.

Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- Según las historias de usuarios brindadas en el caso práctico, crear los casos de prueba.
- Realice la configuración del entorno de pruebas usando maven, java, selenium web driver y cucumber según lo revisado en clase

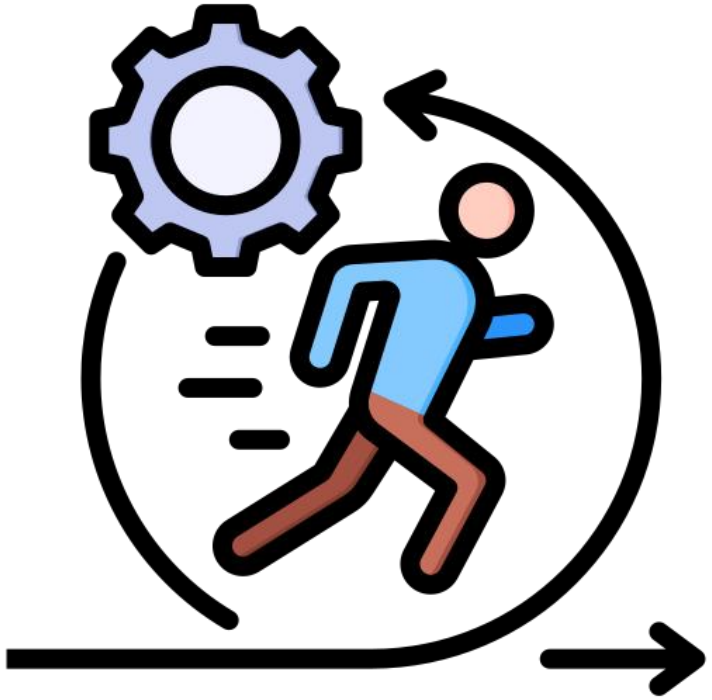
idat

Logro de aprendizaje

Crear casos de prueba a partir de las historias de usuario, empleando los conceptos, roles y ceremonias de SCRUM.

idat

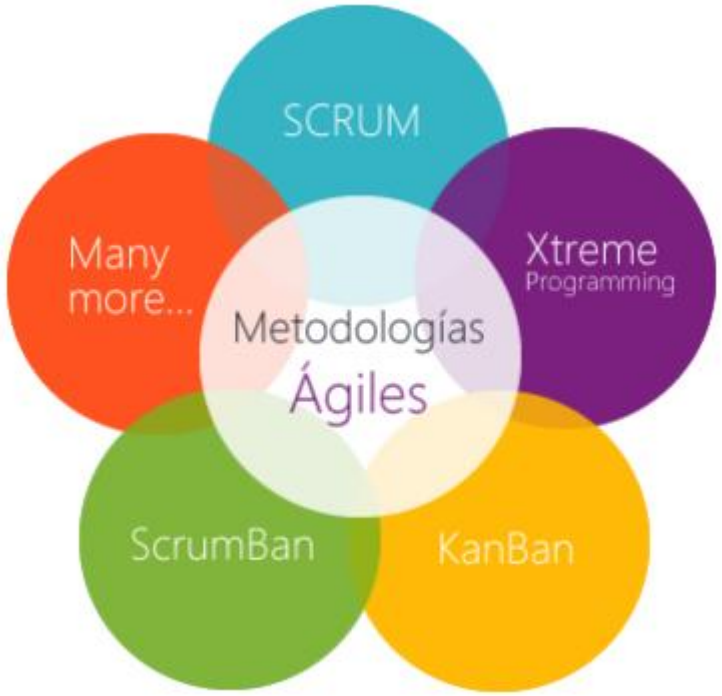
¿Qué es AGILE?



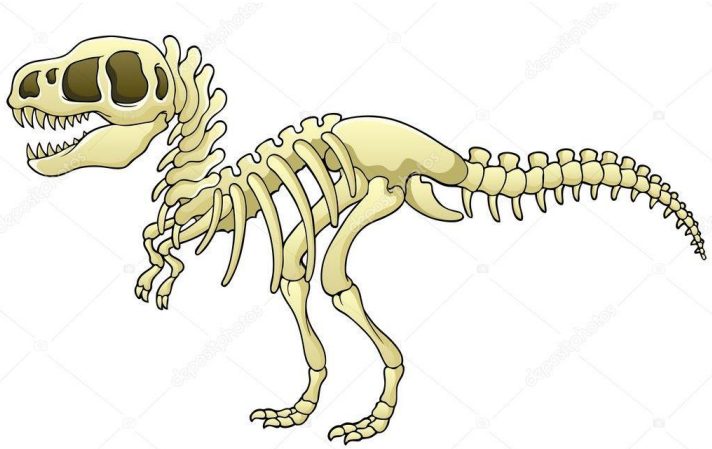
idat

idat

Qué son las metodologías ágiles y su aplicación en el mercado actual



- NO soy un gurú de las metodologías ágiles, sólo me adapto.



idat

Llega el cambio…

El negocio cambia. La tecnología cambia. El equipo cambia.

El problema no es el cambio en sí, porque el cambio va a ocurrir; el problema es la incapacidad para adaptarse al cambio cuando llega.

eXtreme Programming

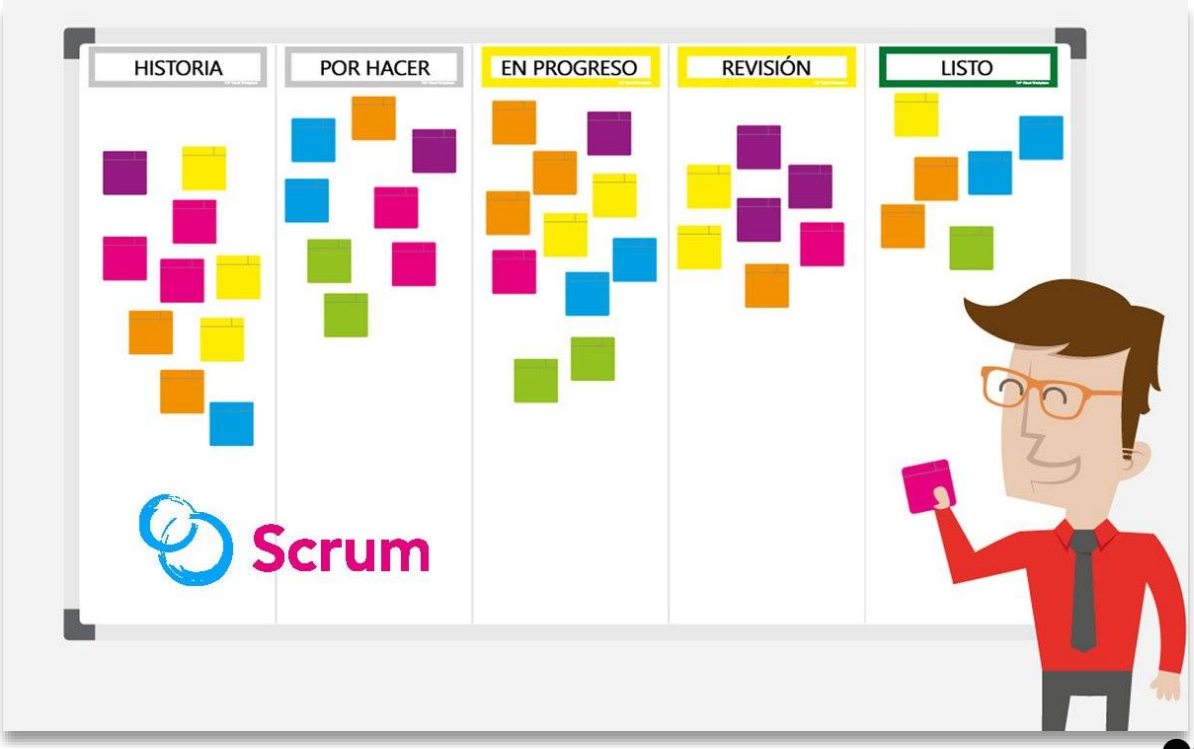
Estamos descubriendo mejores maneras de hacer software

Firmantes del Manifiesto Ágil 2001

idat

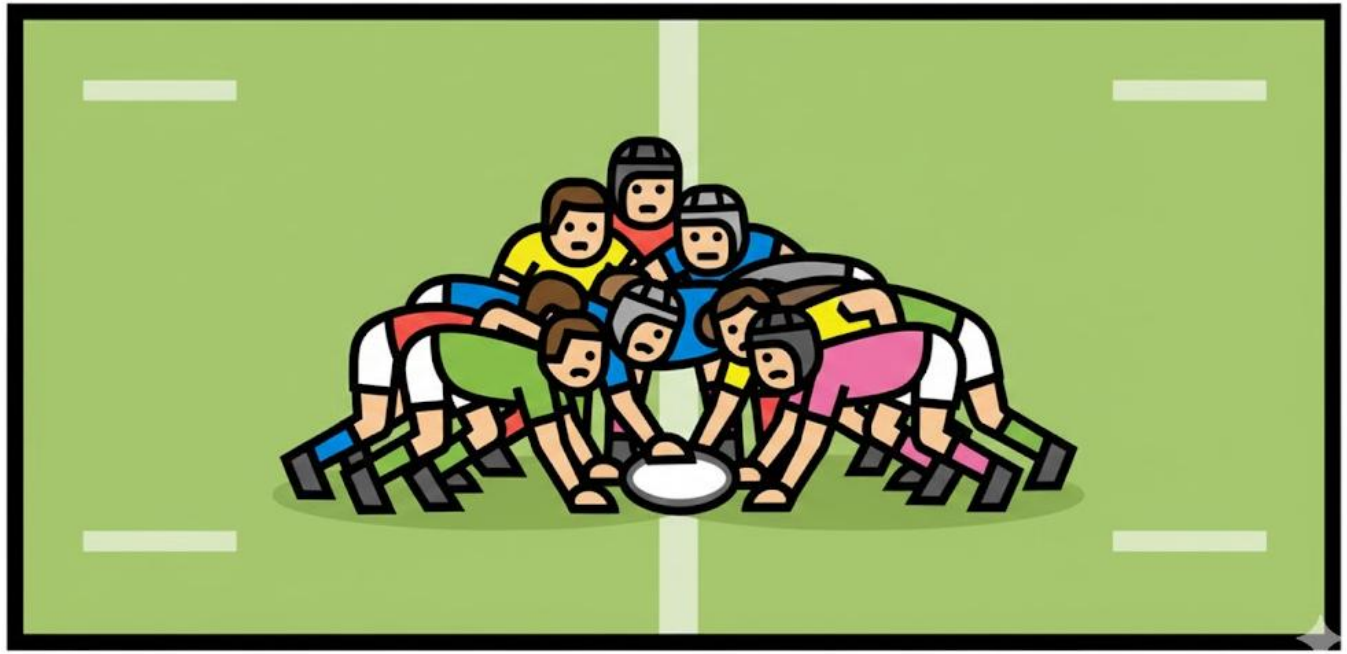
¿Qué es Scrum?

"SCRUM es un marco ágil de trabajo, utilizado en el desarrollo de proyectos de software y que permite obtener el mejor resultado posible en la gestión de un proyecto"



idat

Etimología



“Un scrum es un agrupamiento en Rugby, todos los integrantes del conjunto actúan para avanzar con la pelota”

idat

Características

Marco de trabajo ágil:

- Colaboración estrecha con el cliente
- Predisposición y respuesta al cambio
- Desarrollo incremental con entregas frecuentes
- Comunicación verbal directa
- Motivación, compromiso y responsabilidad del equipo por la autogestión
- Simplicidad de procesos (sólo artefactos necesarios)
- Evitar la burocracia innecesaria

Factores claves en Scrum

- Equipos autoorganizados que toman de decisiones.
- Responsabilidad y autodisciplina
- Trabajo centrado en el compromiso de desarrollo
- Información, transparencia y visibilidad en el desarrollo del proyecto
- Fases de desarrollo solapadas
- La incertidumbre como elemento consustancial y asumido en el entorno y cultura de la organización
- Difusión y transferencia del conocimiento
- Control sutil

idat

Los Sprint

- En Scrum los proyectos avanzan en iteraciones llamadas Sprint de 2–4 semanas de duración (desarrollo de funcionalidad)
- Dentro de cada Sprint, SCRUM gestiona la evolución del proyecto mediante reuniones breves de seguimiento en las que se revisa el trabajo realizado desde el hito anterior y los planes para el hito siguiente
- Las reuniones de seguimiento de cada Sprint deben ser diarias

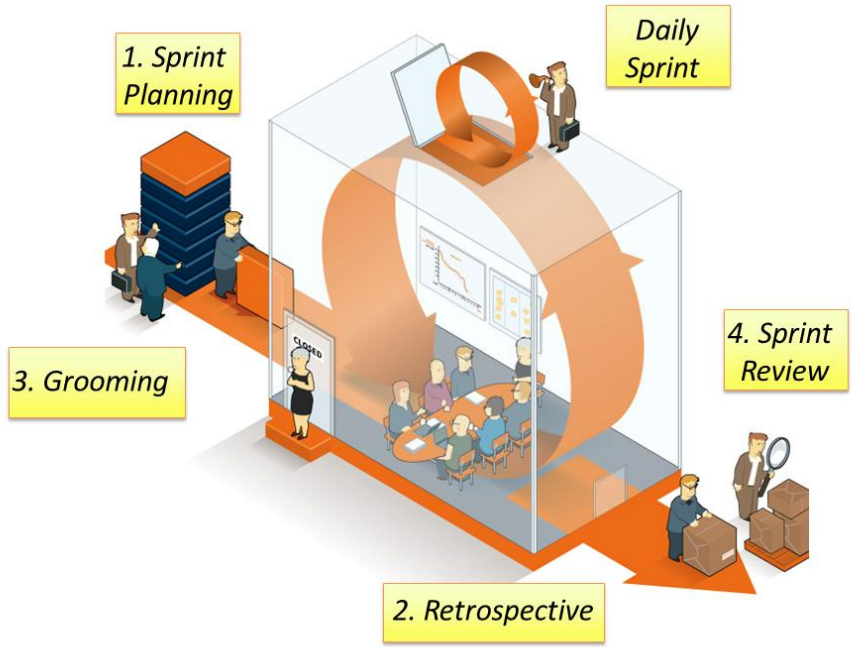
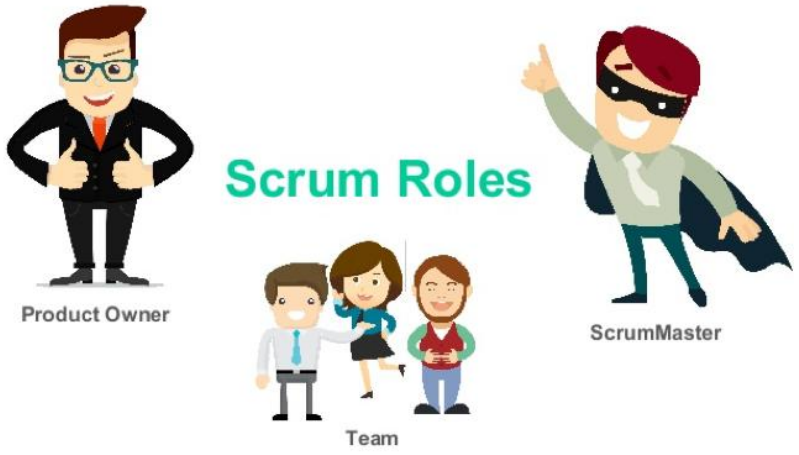
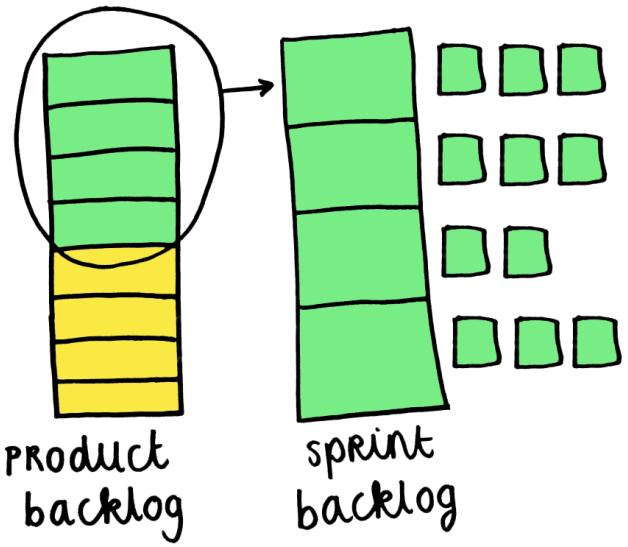


Image from : SoftHouse

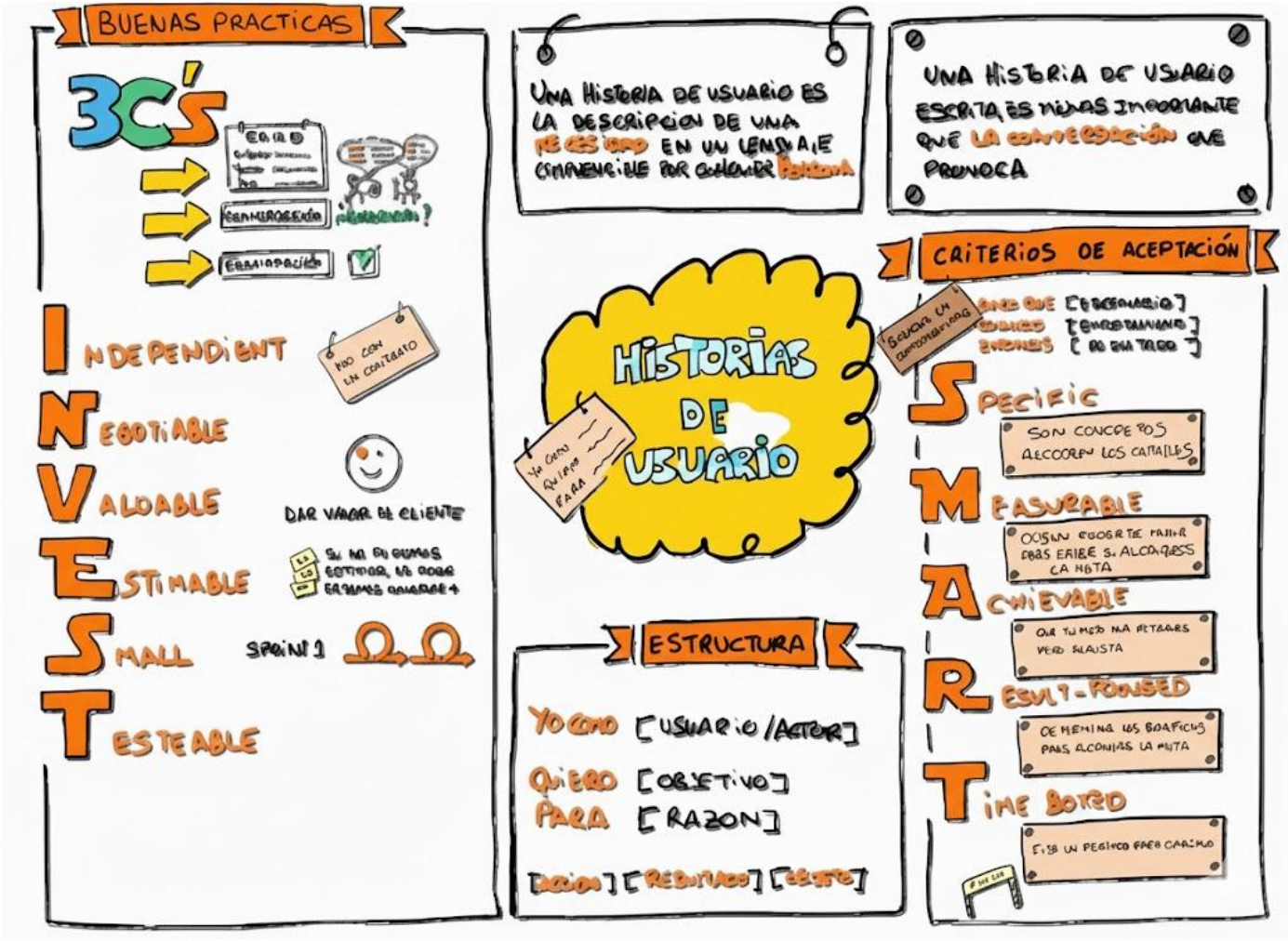
Scrum Framework (componentes)





idat

Historias de Usuario



idat

Historias de Usuario

La ingeniería de requisitos dispone de dos medios de comunicación para recoger y transmitir requisitos, cada uno con sus características:

1. **Comunicaciones escritas.**
   - Registran la información de forma permanente.
   - Son más fáciles de compartir con grupos y personal remoto.
   - Se pueden pensar y revisar bien y de forma completa.
   - Son más susceptibles de malinterpretarse.

2. **Comunicaciones verbales.**
   - Permiten recibir feedback de inmediato.
   - Son dinámicas; la conversación se adapta para maximizar su eficiencia.
   - Se adaptan con facilidad a nuevos desarrollos.
   - Generan ideas nuevas.
   - Permiten alcanzar comprensión y claridad comunes con menos esfuerzo.

idat

Las 3Cs

Las historias de usuario son una herramienta que agiliza la administración de requisitos, reduciendo la cantidad de documentos formales y tiempo necesarios.
Forman parte de la fórmula de captura de funcionalidades definida en 2001 por Ron Jeffries de las tres Cs:

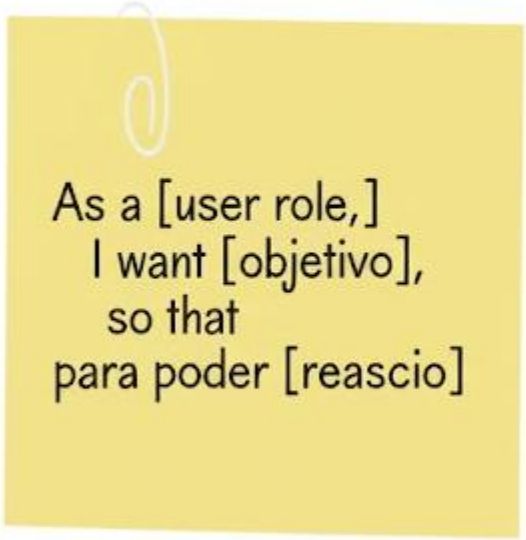
idat

Las 3Cs

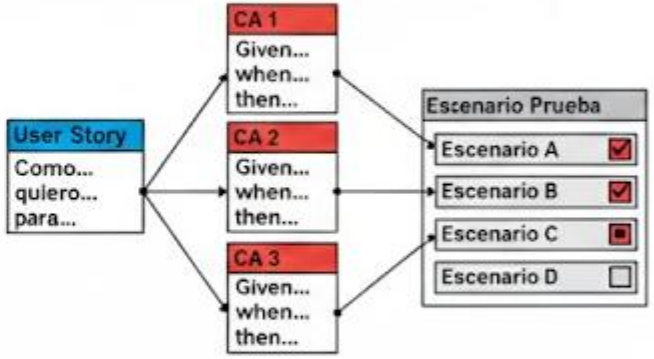
- **Card:** cada historia de usuario se reduce hasta hacerla fácil de memorizar y de sintetizar en una tarjeta o post-it. La tarjeta sirve como recordatorio y promesa de una conversación posterior.
- **Conversation:** el equipo de desarrollo y el propietario del producto añaden criterios de aceptación a cada historia poco antes de su implementación. Los cambios son bienvenidos en agilidad, por lo que no tiene sentido profundizar en estos detalles antes. La situación puede variar mucho desde el momento en el que se sintetiza la funcionalidad en la tarjeta hasta que se implementa.
- **Confirmation:** el propietario del producto o usuario de negocio confirma que el equipo de desarrollo ha entendido y recogido correctamente sus requisitos revisando los criterios de aceptación. A veces se pueden presentar transformados en escenarios de pruebas.

Las 3Cs

Card → Conversation → Confirmation



Sprint Planning
Refinement



idat

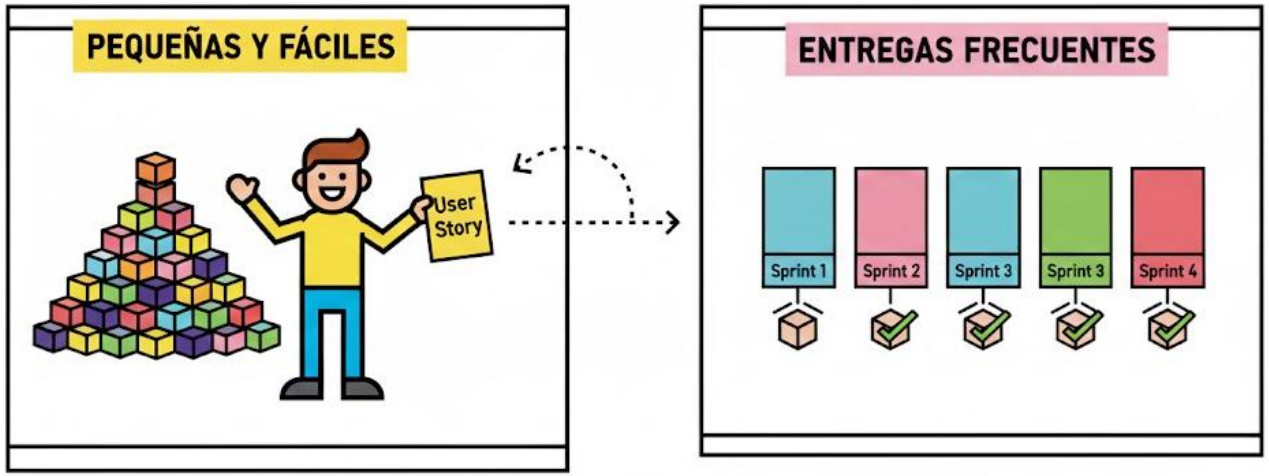
Ventajas de las Historias de Usuario

- Proporcionan la documentación necesaria fomentando a la vez el debate.
- Fomentan la colaboración entre todos interesados y el equipo ágil.
- Se escriben en el lenguaje del usuario, manteniendo así una relación cercana con el cliente.
- Involucran y captan al cliente para el proceso y para el producto.
- Por su naturaleza son independientes.
- Facilitan la planificación e implementación.

idat

Ventajas de las Historias de Usuario

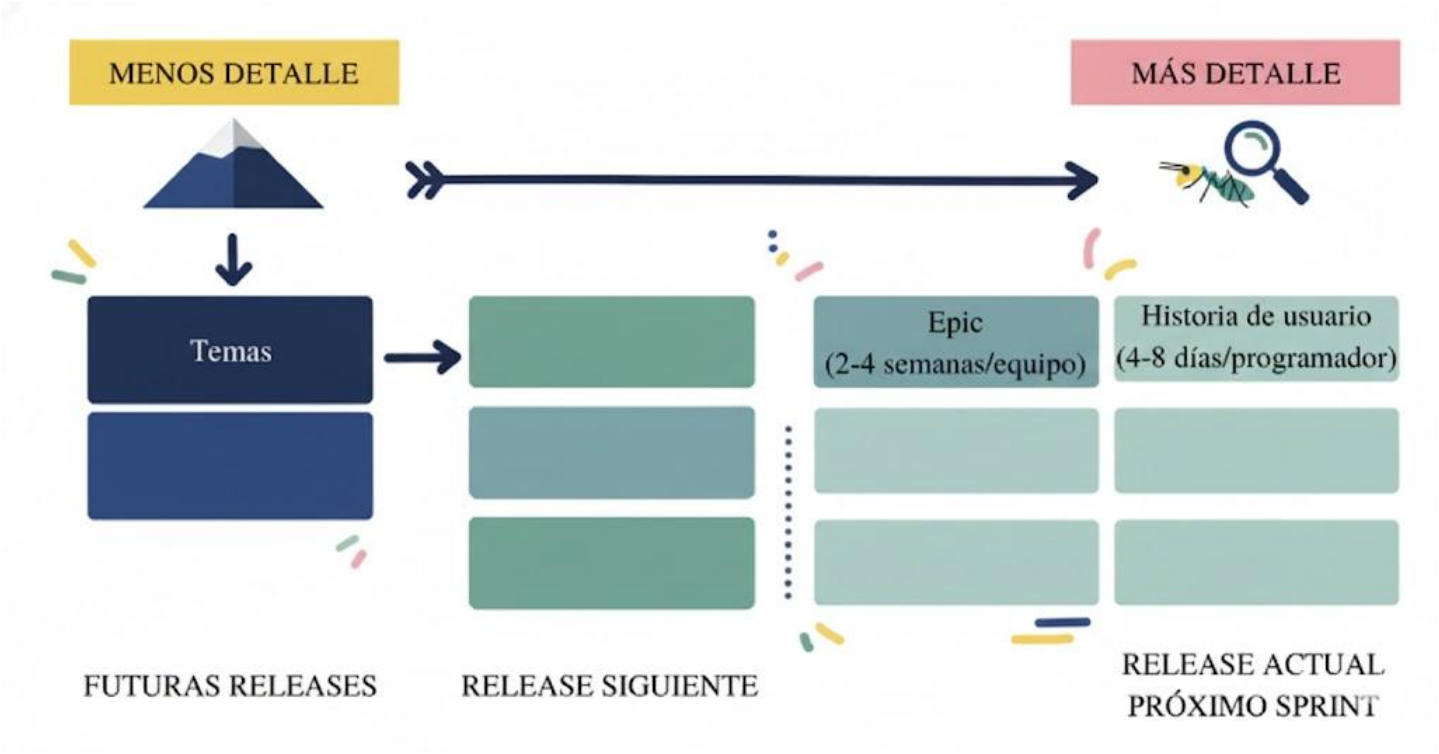
- Son ideales para proyectos con requisitos volátiles o no muy claros.
- Fomentan aplazar los detalles no imprescindibles.
- Son pequeñas y por tanto fáciles trabajar.
- Permiten dividir los proyectos en pequeñas entregas.
- Permiten estimar fácilmente su esfuerzo de desarrollo.
- Funcionan para el desarrollo iterativo, ya que al ser pequeñas representan requisitos del modelo de negocio que pueden implementarse en poco tiempo (días o semanas).
- Necesitan poco mantenimiento.



idat

De Temas y Épicas a Tareas

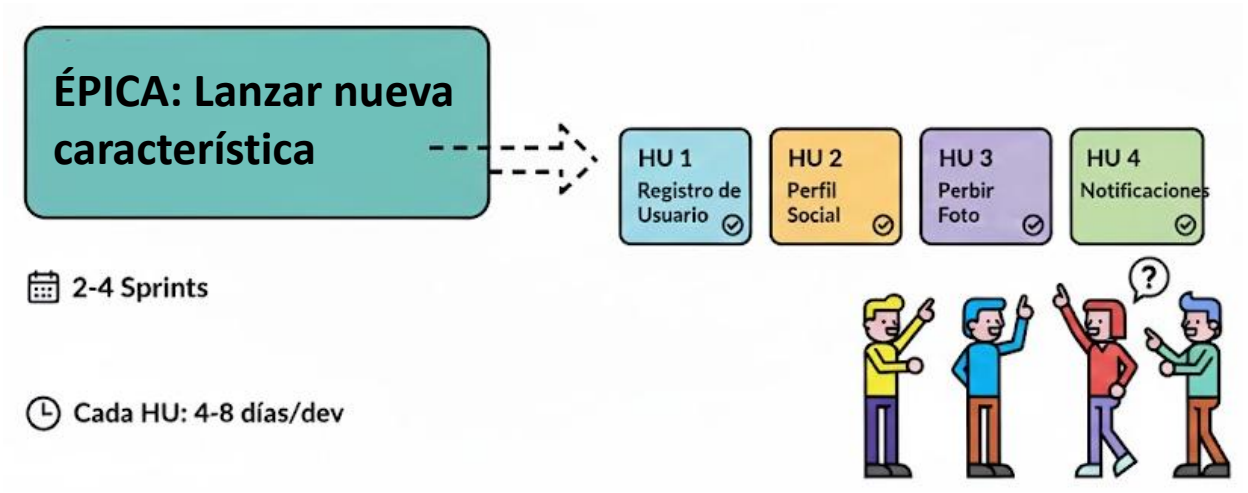
En la jerarquía de los requisitos ágiles y por encima de las historias de usuario se encuentran las epicas y los temas. Tienen en común que todos se centran en describir qué se va a construir.



idat

De Temas y Épicas a Tareas

- **Épica:** una historia de usuario de gran tamaño o alta granularidad, y que tiene por tanto un mayor grado de incertidumbre. Marcar una historia como épica implica que no puede completarse de una sola vez o en un único sprint. Lo normal es que el equipo de desarrollo lo descomponga cuando se acerque el momento de su implementación. Las historias de usuario resultantes estarán íntimamente relacionadas y su menor tamaño permitirá gestionarlas de forma ágil, estimando mejor el tiempo requerido para completarlas y siguiendo su avance con detalle.

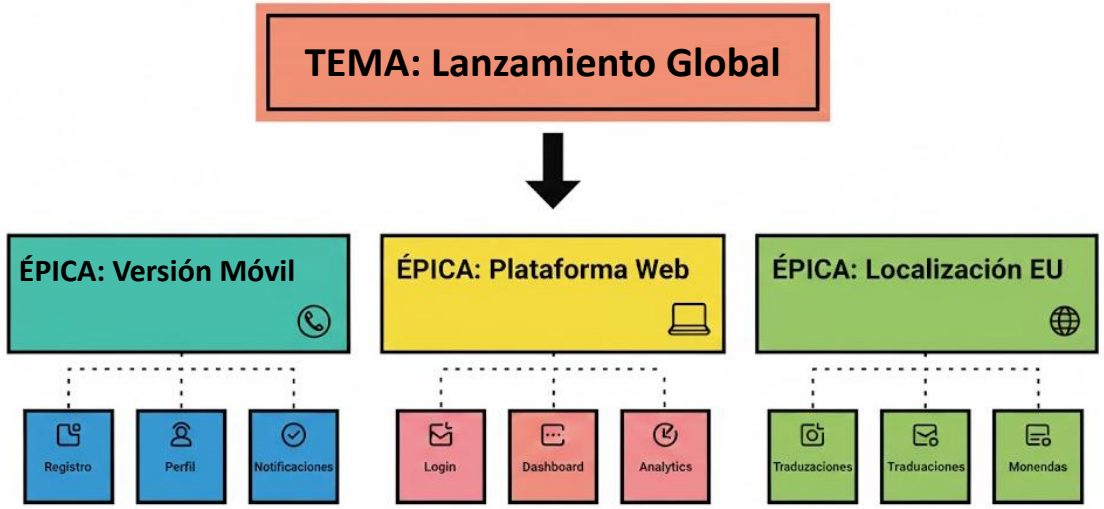


idat

De Temas y Épicas a Tareas

- **Tema:** una colección de épicas e historias de usuario relacionadas que describen un sistema o subsistema. Se trata de un elemento que forma parte de la visión del producto, más que una funcionalidad.

  Por ejemplo: en un sistema de software para gestión contable, el conjunto de épicas «altas, bajas y mantenimiento de clientes», «facturaciones puntuales y recurrentes», «consultas de navegación y acciones de fidelización», «pedidos» y «devoluciones» se podrían denominar como el tema de la «gestión de clientes».



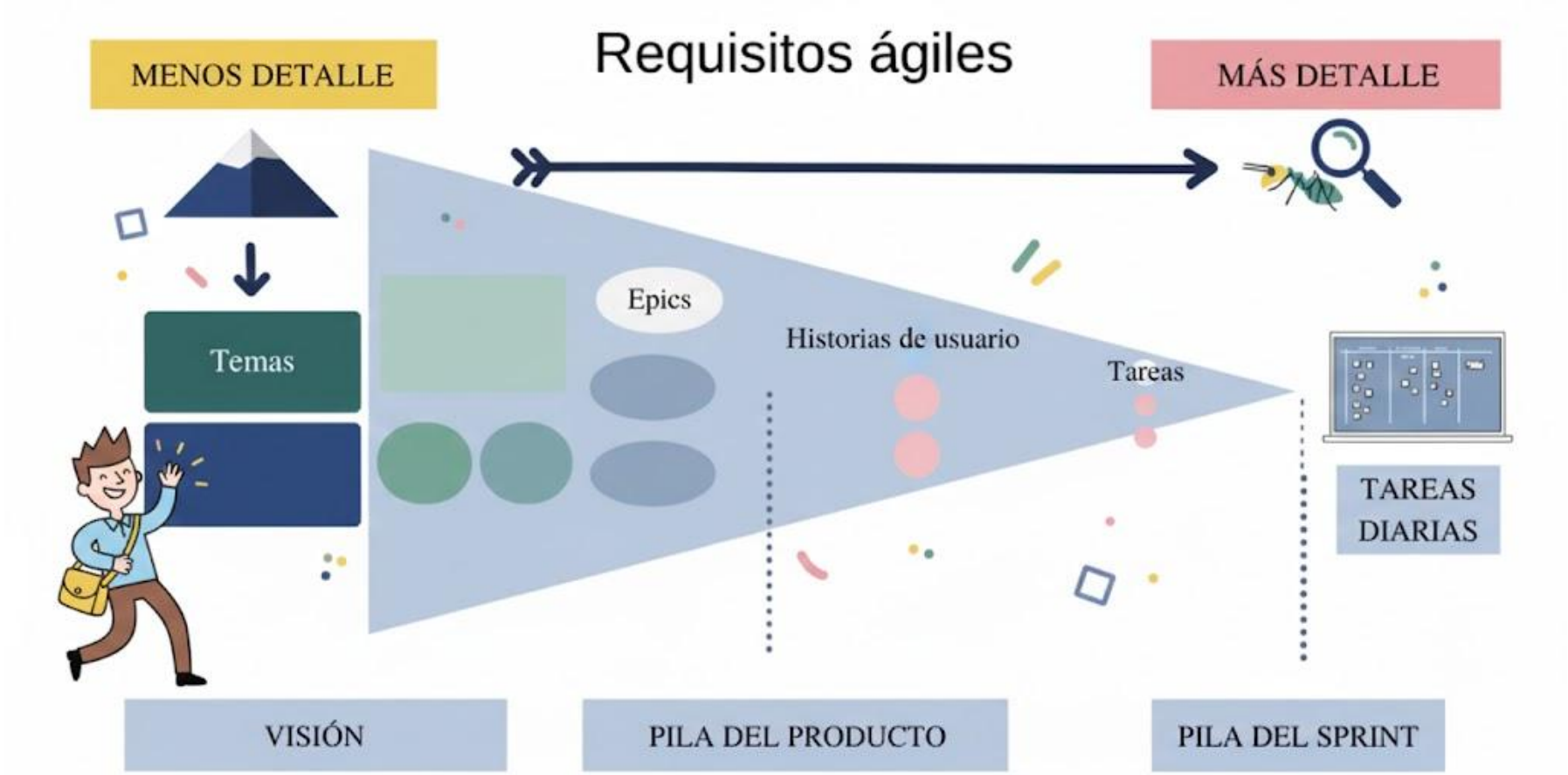
idat

De Temas y Épicas a Tareas

- **Tareas:** están por debajo de las historias de usuario. Describen cómo construir en lugar de qué. Resultan de la descomposición de las historias de usuario en unidades de trabajo adecuadas pare gestionar y seguir el avance de su ejecución.

idat

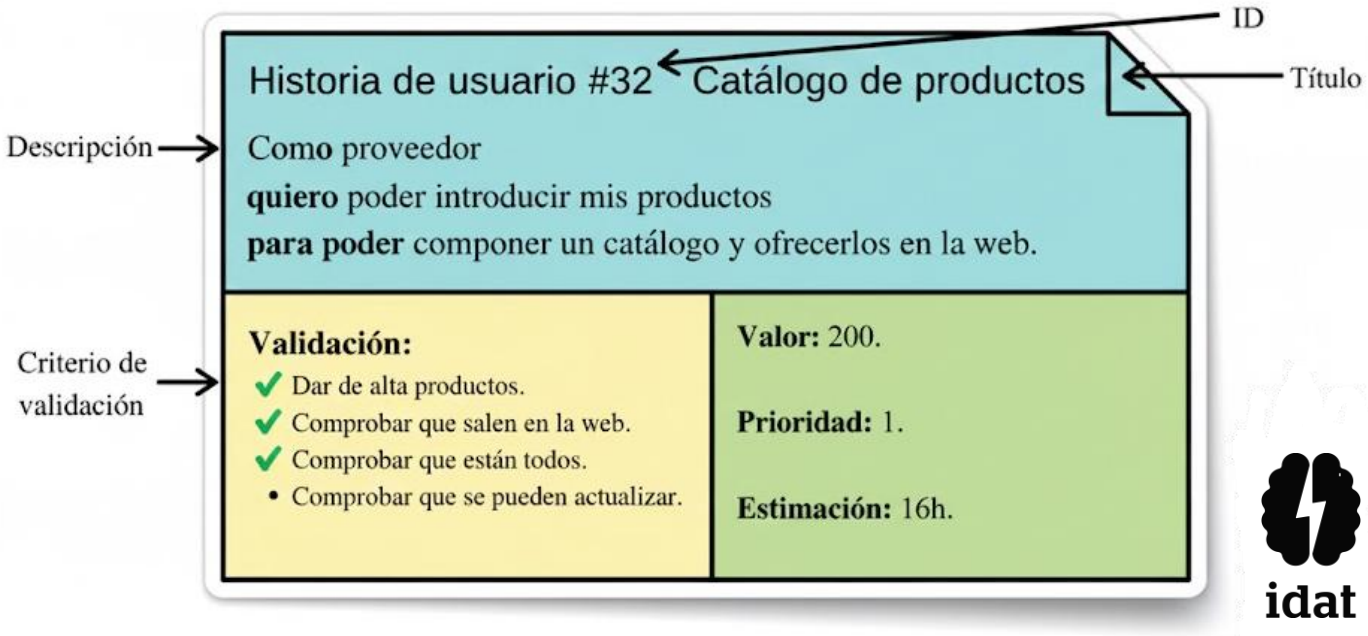
Los cuatro niveles de tamaño con que trata los requisitos la gestión ágil



idat

Información necesaria en las Historias de Usuario

Para decidir qué información incluir en una historia de usuario es preferible no adoptar formatos rígidos. Los resultados de scrum y agilidad no dependen de las formas, sino de la institucionalización de sus principios y la implementación adecuada a las características de la empresa y del proyecto. Por tanto, aparte de tres campos que se consideran necesarios, se puede incluir cualquier campo que proporcione información útil para el proyecto. Recordemos que el objetivo de las historias de usuario es construir un entendimiento compartido.



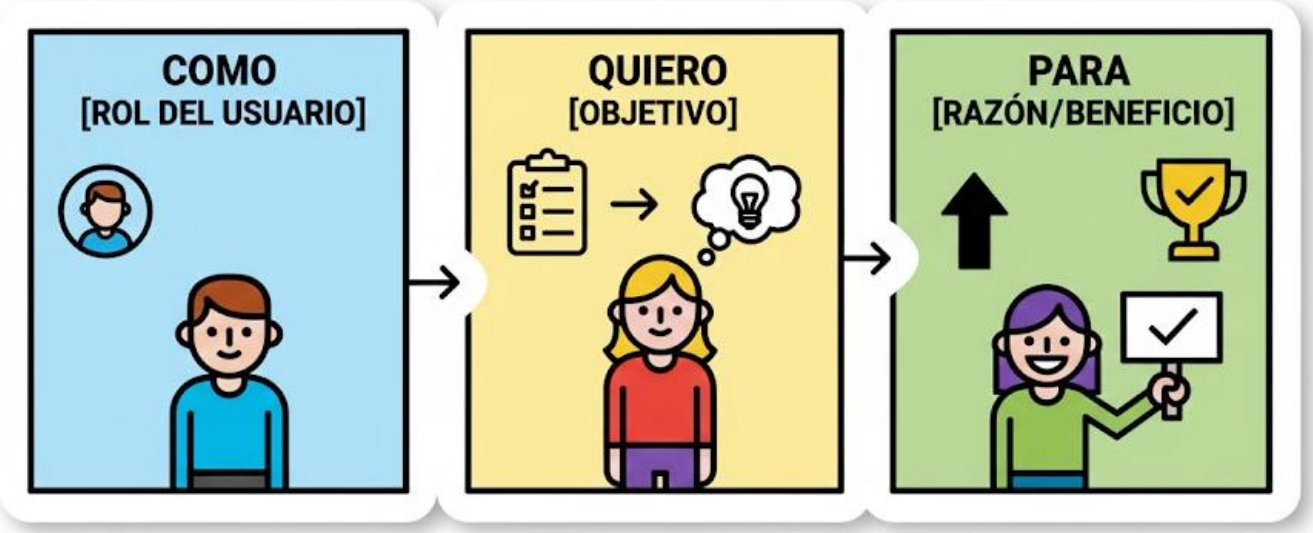
idat

Campos esenciales

Los campos que se consideran más necesarios para describir las historias de usuario son:

- **Descripción:** síntesis de la historia de usuario. El estilo puede ser libre pero debe responder a tres preguntas: ¿quién se beneficia? ¿qué se quiere? y ¿cuál es el beneficio?

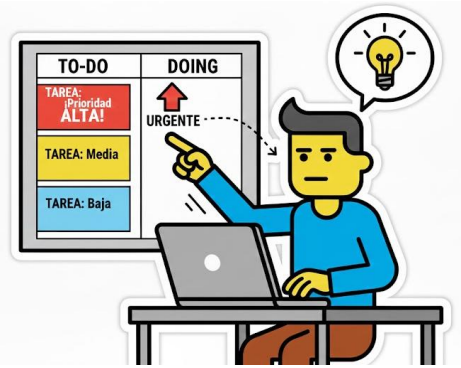
  Mike Cohn recomienda seguir el siguiente patrón para garantizar que la funcionalidad esté descrita a un alto nivel y de manera breve.



Campos esenciales

- **Estimación:** aproximación del esfuerzo necesario (en tiempo ideal) para implementar la historia de usuario. Puede estimarse usando unidades de desarrollo (puntos de historia), si el equipo lo prefiere y está familiarizado con este sistema.

- **Prioridad:** se indica siguiendo un sistema que permita establecer el orden de implementación de las historias.



idat

Otros campos

Dependiendo del tipo de proyecto, el funcionamiento del equipo y la organización, pueden ser aconsejables otros campos como:

- **ID:** identificador único de la historia de usuario, funcionalidad o trabajo.
- **Titulo:** título descriptivo de la historia de usuario.
- **Valor de negocio:** valor (normalmente numérico) que aporta la historia de usuario al cliente o usuario. El objetivo del equipo es maximizar el valor y la satisfacción percibida por el cliente en cada iteración. Este campo servirá junto con la estimación para decidir la prioridad de implementación.
- **Criterio de aceptación:** pruebas de aceptación consensuadas con el cliente o usuario. A veces se transforman en pruebas que el código debe superar para dar como finalizada la implementación.

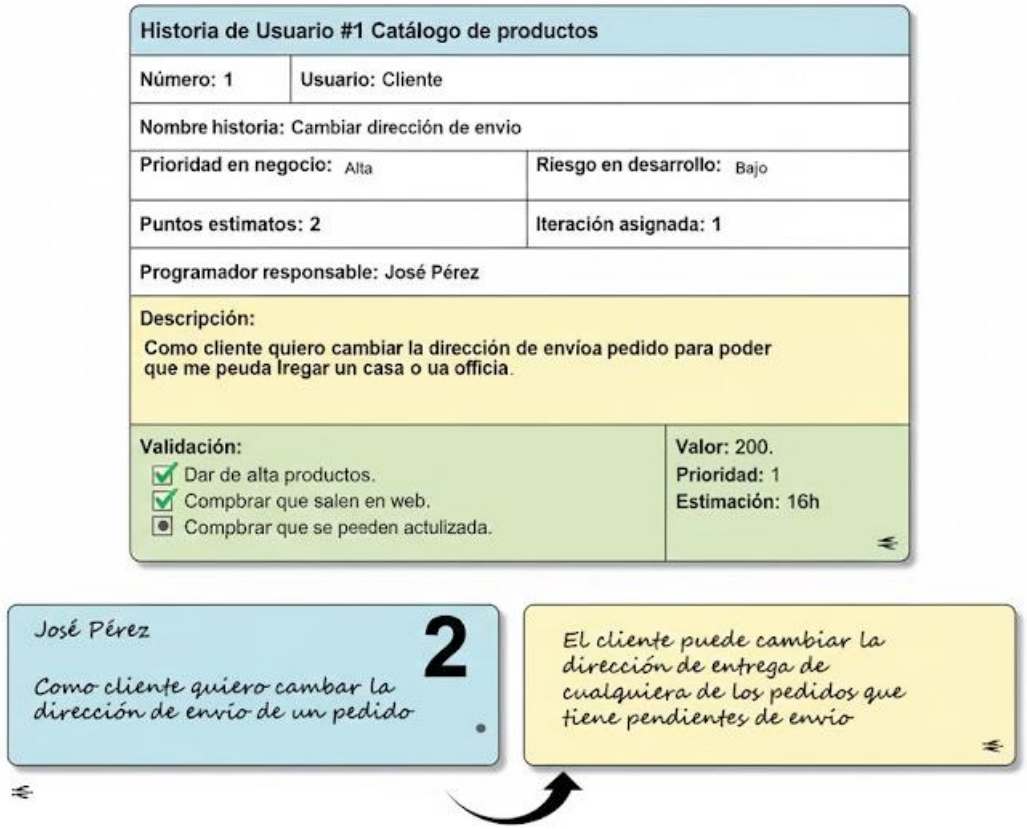
Otros campos

- **Requerimiento no funcional:** cualidades generales y restricciones, como la usabilidad o la seguridad, que afectan a aplicaciones y sistemas enteros, y por tanto a las historias de usuario individuales.
- **Definición de hecho (en inglés DoD, Definition of Done):** incluye las actividades o criterios necesarios para dar por terminada una historia de usuario (desarrollada, probada, documentada…), según lo convenido por el propietario de producto y el equipo.
- **Dependencias:** una historia de usuario no debería depender de otra, pero a veces es necesario mantener la relación. Este campo contendría los identificadores de otras historias de las que depende.
- **Persona asignada:** cuando queramos sugerir la persona que pueda implementar la historia de usuario. Recordar que en scrum el equipo se autogestiona y es quien distribuye y asigna las tareas.

idat

Otros campos

- **Sprint:** puede ser útil para organización del propietario del producto incluir el número de sprint en el que se prevé construir la historia.
- **Riesgo:** riesgo técnico o funcional asociado a la implementación de la historia de usuario.
- **Módulo:** módulo del sistema o producto al que pertenece.
- **Observaciones:** para enriquecer o aclarar la información, o cualquier otro uso necesario.



idat

A considerar…

Si bien las historias de usuario son lo bastante flexibles como para describir la **funcionalidad** de la mayoría de los sistemas, **no son apropiadas para todo**. Si por cualquier razón se necesita expresar algún requisito de manera diferente, es recomendable hacerlo.

Por ejemplo: la maquetación de una interfaz se suele describir con capturas de pantalla. Sería la mejor manera de transmitir el diseño que queremos darle a una aplicación.

idat

Descripción

El pensamiento de las personas se estructura siguiendo una narrativa, una historia; así es como entendemos el mundo. Es también una manera eficaz de retener conocimiento y tomar decisiones. Al usar personajes y empatizar con ellos podemos entender su punto de vista y priorizar de manera más acertada.

Una técnica útil en este sentido es la creación de user personas, muy utilizadas en el ámbito de la **user experience** (UX, «experiencia de usuario» en español). El objetivo es comprender el punto de vista de los usuarios: construir una serie de personajes de ficción que representan, mediante arquetipos, una muestra realista de la audiencia clave de nuestra web o aplicación.

Estas user personas servirán como «rol del usuario» en el patrón que vimos antes para la descripción de la historia:

Como [rol del usuario], **quiero** [objetivo], **para poder** [beneficio].

idat

Descripción

- **Como [rol del usuario]:** ¿para quién estamos construyendo? No nos referimos a un cargo o un rol profesional, sino a la persona tras la necesidad descrita, según esté representada en su user persona. El equipo debe tener un entendimiento compartido de quién es, cómo trabaja, cómo piensa y siente; en definitiva, sentir empatía.
- **Quiero [objetivo]:** ¿qué es lo que está intentando conseguir realmente? Es importante recordar que en este campo se explica su intención, no la funcionalidad que usará.
- **Para poder [beneficio]:** ¿cómo encaja su necesidad inmediata en lo que le rodea? ¿cuál es el beneficio general que está tratando de lograr? ¿cuál es el gran problema que necesita resolver? Se trata de un beneficio que va relacionado con la visión del producto.

Personajes

Facilitan mantener las conversaciones centradas en los usuarios, dándoles un nombre y una personalidad para empatizar con el grupo de personas al que representan. Si el público objetivo de nuestra web tiene unas características determinadas, hacer su user persona y referirnos a ella por su apodo nos permite imaginar mejor cómo interactuará con el sistema, ayudando al equipo a tomar decisiones más acertadas y ofrecer soluciones más adecuadas a sus necesidades reales.

Se suele representar a los usuarios en una ficha tipo tríptico incluyendo, por ejemplo:

- **Nombre y apodo:** una manera rápida de identificar y reconocer al personaje.
- **Datos demográficos:** ayudan a clasificar al personaje dentro de un segmento de mercado.
- **Descripción:** un párrafo breve que nos ayude a conocer al personaje.
- **Objetivos:** necesidades del usuario que distinguen su comportamiento del resto.

Personajes

Roberto García
“El scrumista”

Datos demográficos: padre de familia de 40 años que utiliza la tecnología de forma habitual

Descripción: afable e inquieto, gran lector de libros y blogs sobre agilidad y scrum. Es miembro de varios grupos de interés del mundo ágil.

Objetivos: como Scrum Master quiere estar al día sobre prácticas ágiles y formarse a través de cursos y comunidades de práctica.

idat

Valor de negocio

- El campo «valor» de una historia de usuario representa el valor de negocio que aportará una vez realizada. Aporta información más profunda que la razón de la historia, pues está ligado a la lógica del negocio. Una manera de estimarlo es considerar cuánto estaría dispuesto a pagar el cliente por la funcionalidad.
- Tal como funcionan scrum y otras metodologías ágiles, entregar valor al cliente significa centrarnos en resolver los problemas de alguien o en dar beneficios a alguien. Es muy importante conocer el valor de negocio para priorizar correctamente y focalizar al equipo. También existen otros factores a tener en cuenta, por lo que valor y prioridad son dos informaciones independientes.
- El valor se mide con una escala arbitraria y normalmente numérica. Ha de ser una con la que el propietario del producto y los usuarios se sientan cómodos, por ejemplo la serie de Fibonacci, o series de números naturales como del 1 al 10, del 1 al 100 o del a 1.000.000.

idat

Estimación

- Como hemos visto, es parte de la información esencial en una historia de usuario. Estimar el esfuerzo ayuda al propietario de producto a priorizar y al equipo a decidir qué historias de la pila caben en el sprint.
- Un hecho a considerar es que la incertidumbre para estimar historias de usuario pequeñas es mucho menor que para historias grandes. Además, aunque el tamaño de las historias crece linealmente, la incertidumbre lo hace exponencialmente.

Prioridad

Aunque todas las historias de usuario puedan ser importantes, para focalizarnos de forma eficiente hay que destacar las que den más valor al sistema. El Product Owner debe asignar un valor a cada historia que intervenga en el sistema de priorización, teniendo en cuenta las siguientes variables:

- Beneficios de implementar la funcionalidad.
- Pérdida o costo que se derive de posponer su implementación.
- Riesgos de implementarla.
- Coherencia con los intereses del negocio.
- Valor diferencial con respecto a productos de la competencia.

Un aspecto a tener en cuenta es que la definición de valor puede ser distinta para cada cliente. Es recomendable utilizar una escala cualitativa, con un significado intrínseco, que aporte más información que si la prioridad es alta, media, o baja.

Prioridad

Es el caso de la técnica MoSCoW, en la que el usuario responsable de asignar la prioridad es consciente del efecto real que producirá su elección. Su finalidad es alcanzar un entendimiento común del proyecto entre cliente y equipo, en concreto sobre la importancia de cada historia de usuario.

idat

Prioridad

La clasificación es:

- **Must have, es necesario:** la funcionalidad debe estar implementada en la solución, si no ésta fallará o no podrá considerarse un éxito.
- **Should have, es recomendable:** se debería implementar, ya que es una funcionalidad de alta prioridad. La solución no fallará si no existe, pero debería haber causas justificadas.
- **Could have, podría implementarse:** es deseable, sería conveniente tener esta funcionalidad en la solución, pero dependerá de los tiempos y el presupuesto del proyecto.
- **Won't have, no lo queremos por ahora:** se trata de una funcionalidad de muy baja prioridad o descartada, pero que en el futuro puede ser relevante. En tal caso pasaría a alguno de los otros estados.

idat

A considerar…

Es importante distinguir entre prioridad y valor. Puede que una historia de usuario no tenga ningún valor para el cliente o usuario, pero que esta sea imprescindible y de alta prioridad.

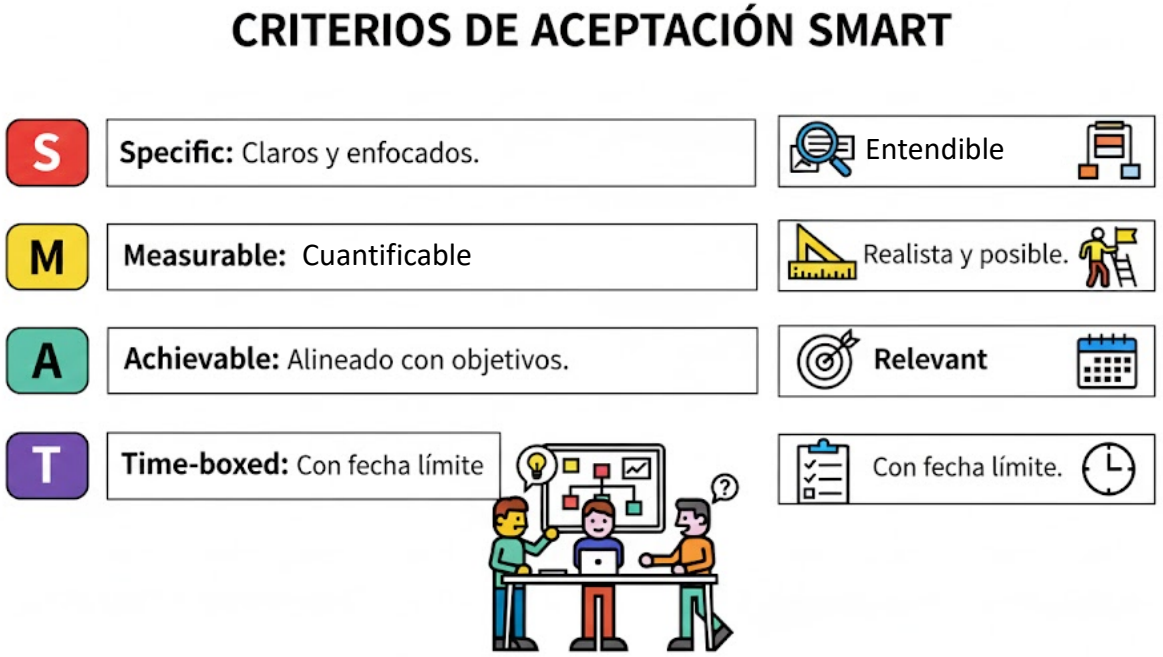
Un caso sería la infraestructura necesaria para la implementación de un software, que no aporta valor al cliente en sí, pero sin la cual no se puede desarrollar ni ejecutar la solución.

idat

Criterios de aceptación

Para medir la calidad de un criterio de aceptación se utiliza el método SMART, por el que se han de ser, en la medida de lo posible:

- **Specific,** específicos.
- **Measurable,** medibles.
- **Achievable,** alcanzables.
- **Relevant,** relevantes.
- **Time-boxed,** limitados en el tiempo.



idat

Criterios de aceptación

Los criterios se suelen expresar en forma de checklist o como un flujo en cuanto se obtienen historias de usuario susceptibles de implementarse, y se refinan en la planificación de sprint. Ayudan al equipo de desarrollo a entender el funcionamiento del producto, de manera que estimarán mejor el tamaño de la historia subyacente.
Durante la fase de desarrollo servirán además de guía para la tomar de decisiones.
Durante la revisión del sprint, el propietario de producto comprobará, a través de estos criterios de aceptación y la definición de hecho (DoR), si cada una de las historias de usuario se puede aceptar y dar por finalizada.

idat

Criterios de aceptación

Los criterios de aceptación pueden escribirse en lenguaje natural, tal como el propietario del producto se expresa, y pueden adoptar diferentes formatos. Por ejemplo:

Comprobar [criterios].

Demostrar [comportamiento esperado].

Verificar que cuando [rol] **hace** [acción] **consigue** [resultado / comportamiento esperado].

Dado que [contexto] **y adicionalmente** [contexto] **cuando** [evento]

entonces [resultado / comportamiento esperado].

idat

Criterios de aceptación

Una opción excelente es escribirlos con la técnica del comportamiento por escenarios propia de **BDD** (Behavior Driven Development) y con **gherkin**, un lenguaje creado para las descripciones de comportamiento de software. La sintaxis de gherkin es la siguiente:

(*Scenario*) **Escenario** [número de escenario] [titulo del escenario]:

(*Given*) **Dado que** [contexto] **y adicionalmente** [contexto],

(*When*) **Cuando** [evento],

(*Then*) **Entonces** [resultado / comportamiento esperado].

idat

Criterios de aceptación

Los elementos de los criterios de aceptación derivados de esta sintaxis son:

- **Número de escenario:** un identificador para el escenario asociado a la historia.
- **Título del escenario:** describe el escenario que define un comportamiento.
- **Contexto:** detalla las condiciones que desencadenan el escenario.
- **Evento:** representa la acción que el usuario ejecuta, en el contexto definido para el escenario.
- **Resultado / comportamiento esperado:** el comportamiento del sistema en esa situación.

```
Scenario: Some determinable business situation
  Given some precondition
    And some other precondition
   When some action by the actor
    And some other action
    And yet another action
   Then some testable outcome is achieved
    And something else we can check happens too
```

idat

Ejemplo de criterios de aceptación escritos en gherkin

Historia de usuario

Como cliente
quiero retirar dinero del cajero
para poder evitar ir al banco a hacer cola.

Criterios de aceptación

Escenario 1: cuenta tiene crédito.

Dado que la cuenta tiene crédito
Y que la tarjeta es válida
Y que el cajero tiene dinero disponible
Cuando el cliente pide dinero
Entonces la cuenta es debitada
Y el dinero es entregado al cliente
Y el cliente recupera su tarjeta

Escenario 2: la cuenta excede el límite negativo acordado con el banco.

Dado que la cuenta excede el límite negativo acordado con el banco
Y que la tarjeta es válida
Cuando el cliente pide dinero
Entonces el cajero muestra un mensaje negando el pedido
Y el dinero no es entregado al cliente
Y el cliente recupera su tarjeta

idat

Historias de usuario de calidad

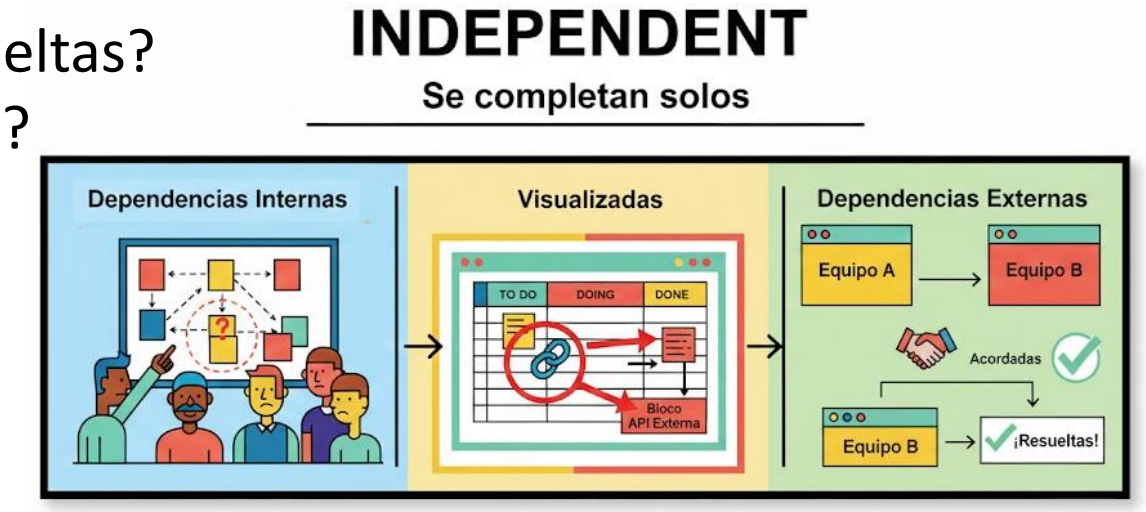
En 2003 Bill Wake desarrolló un método llamado INVEST para asegurar la calidad en la escritura de historias de usuario. El método sirve para comprobar la calidad de una historia de usuario revisando que cumpla una serie de características:

- **Independent,** independiente.
- **Negotiable,** negociable.
- **Valuable,** valiosa.
- **Estimable,** estimable.
- **Small,** pequeña.
- **Testable,** comprobable.

idat

Método INVEST

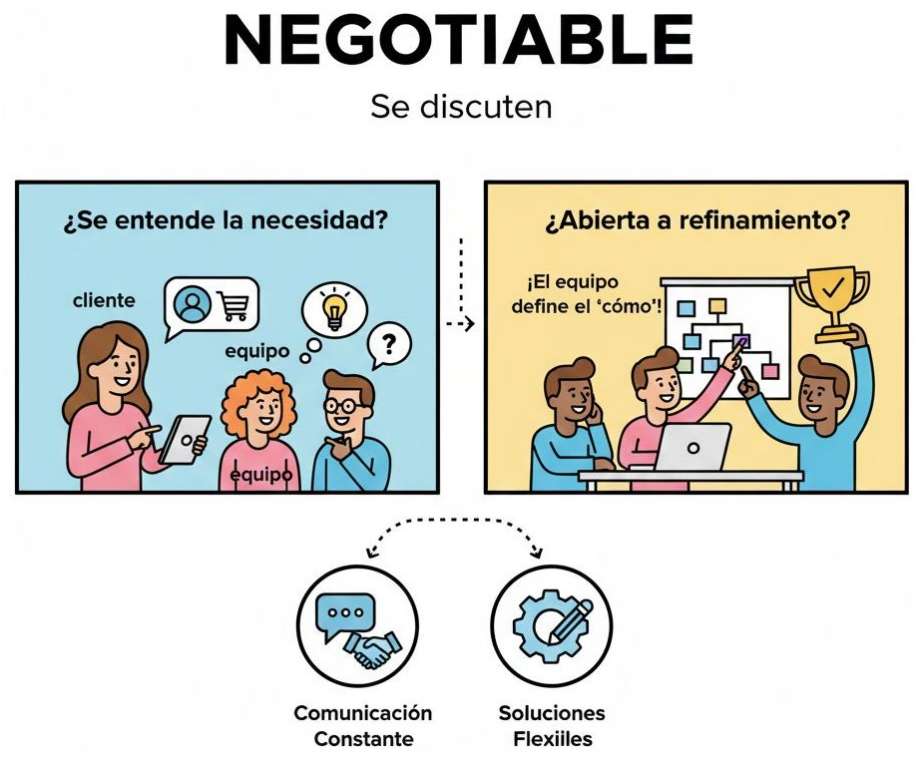
- **Independiente:** poder planificar e implementar las historias de usuario en cualquier orden tiene muchas ventajas y facilita el trabajo posterior del equipo. Para que sea posible, cada historia debería ser independiente. Una forma de reducir las dependencias es combinar historias o dividirlas de manera diferente.
  - Las dependencias internas están resueltas?
  - ¿Las dependencias están visualizadas?
  - ¿Las dependencias externas están acordadas?
  - ¿Las dependencias externas están resueltas?



idat

Método INVEST

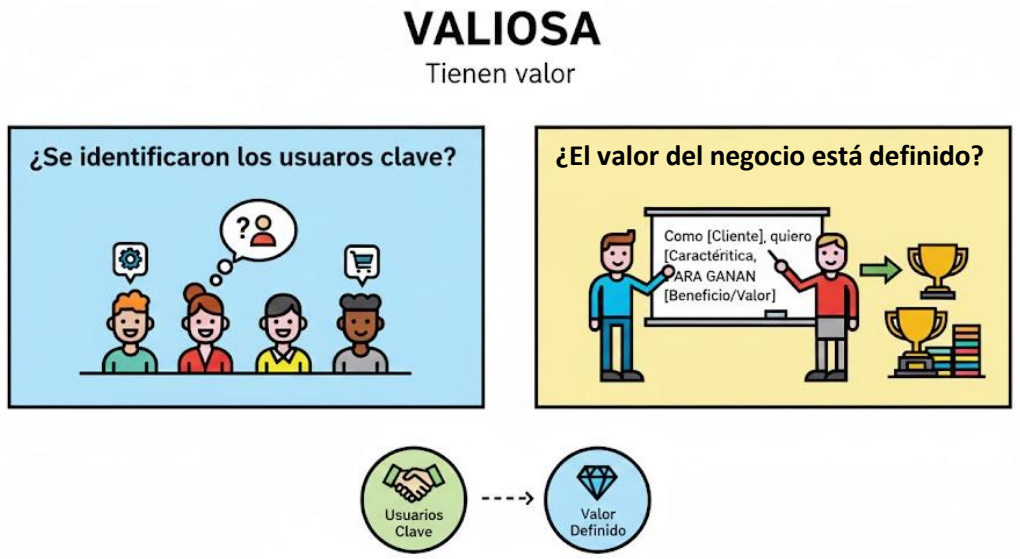
- **Negociable:** una historia de usuario es una descripción corta de una necesidad, sin detalles. Deben ser negociables ya que los pormenores serán acordados con el cliente o el usuario más adelante. Conviene mantener las historias en términos generales o vagos para no limitar estas conversaciones.
  - ¿Se entiende la necesidad a resolver del usuario?
  - ¿La descripción está abierta a futuros refinamientos?
  - ¿El equipo puede definir el cómo?
  - ¿Sólo el equipo formula el cómo?



idat

Método INVEST

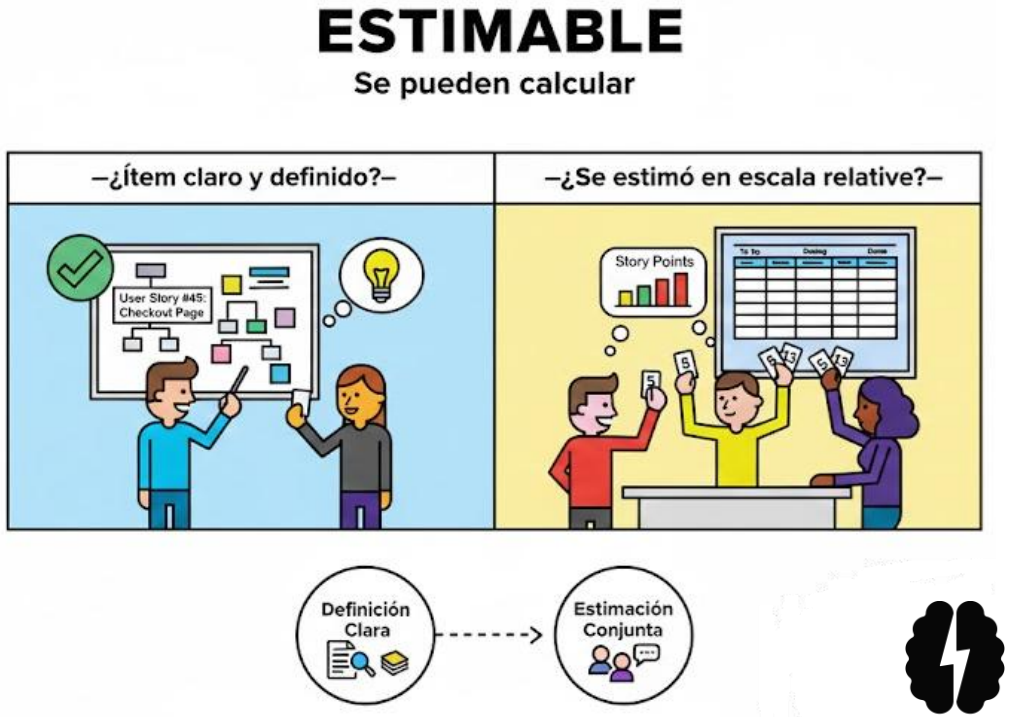
- **Valiosa:** una historia de usuario tiene que aportar valor al cliente o al usuario. Una manera de conseguirlo es que sean ellos quienes la escriban.
  - ¿Se identificaron los usuarios clave?
  - ¿El valor de negocio está definido?
  - ¿La participación en la definición de valor de negocio es la adecuada?
  - ¿El valor de negocio permite negociar?



idat

Método INVEST

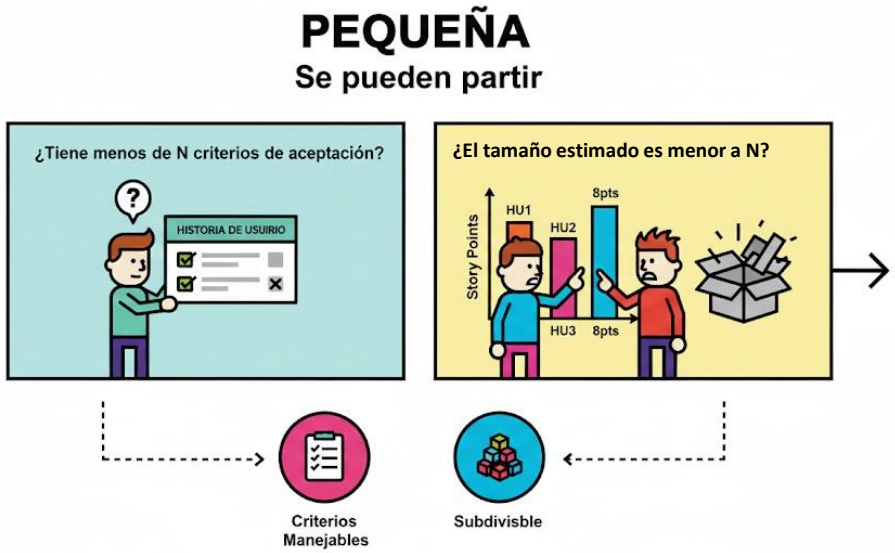
- **Estimable:** una buena historia de usuario debe poder estimarse con precisión suficiente para que el propietario del producto pueda priorizar y planificar su implementación. El equipo es quien suele realizar la estimación, y ésta se verá afectada por el tamaño de la historia, pues a mayor tamaño mayor incertidumbre, y por el conocimiento del equipo sobre la necesidad expresada. En caso de falta de conocimiento, serán necesarias más fases de conversación.
  - ¿El ítem se definió y registró claramente?
  - ¿El equipo está listo para estimar?
  - ¿Se estimó en una escala relativa de referencia?
  - ¿Todo el mundo participó en la estimación?



idat

Método INVEST

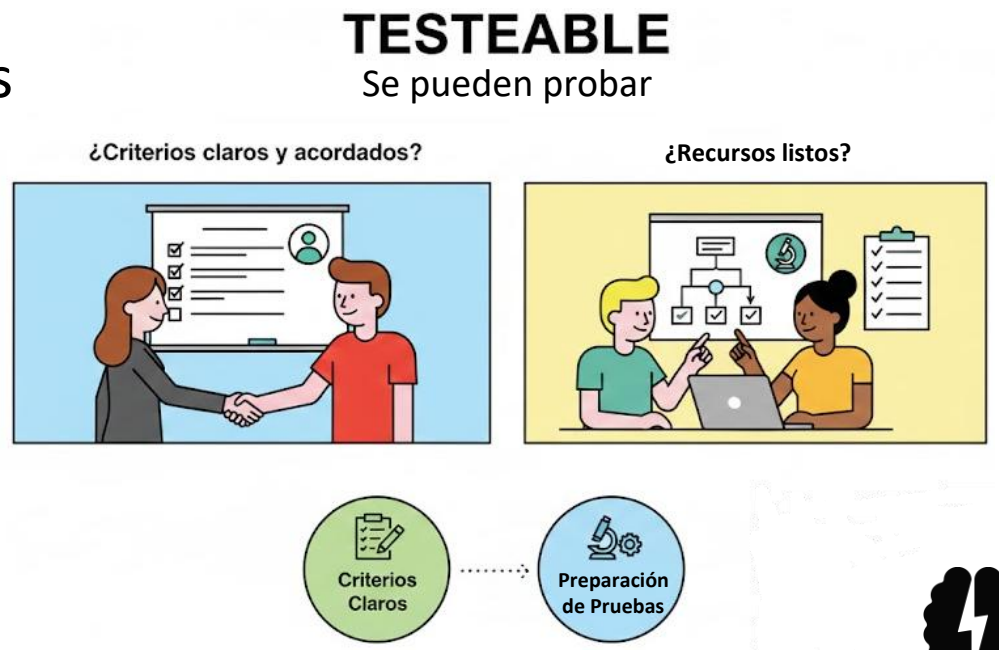
- **Pequeña:** las historias de usuario deberían englobar como mucho unas pocas semanas de trabajo por persona. Incluso hay equipos que las restringen a días por persona. Una descripción corta ayuda a disminuir el tamaño de una historia de usuario, facilitando así su estimación.
  - ¿Tiene menos de <N> criterios de aceptación?
  - ¿El tamaño estimando es menor a <N>?
  - ¿Ya no tiene sentido subdividir más?
  - ¿Ya no hay riesgos de aumento de tamaño?



idat

Método INVEST

- **Comprobable:** la historia de usuario debería poder probarse en la fase de confirmación. Si el cliente o usuario no sabe cómo, significa que la funcionalidad no es del todo clara o que no es valiosa, y si el equipo no puede probarla es imposible que sepa si está terminada o no.
  - ¿Los criterios de aceptación están definidos y claros?
  - ¿Los criterios de aceptación están acordados con los usuarios?
  - ¿Se generó la documentación necesaria?
  - ¿Se acordó la provisión de recursos críticos de prueba?
  - ¿La provisión de recursos críticos de prueba está resuelta?



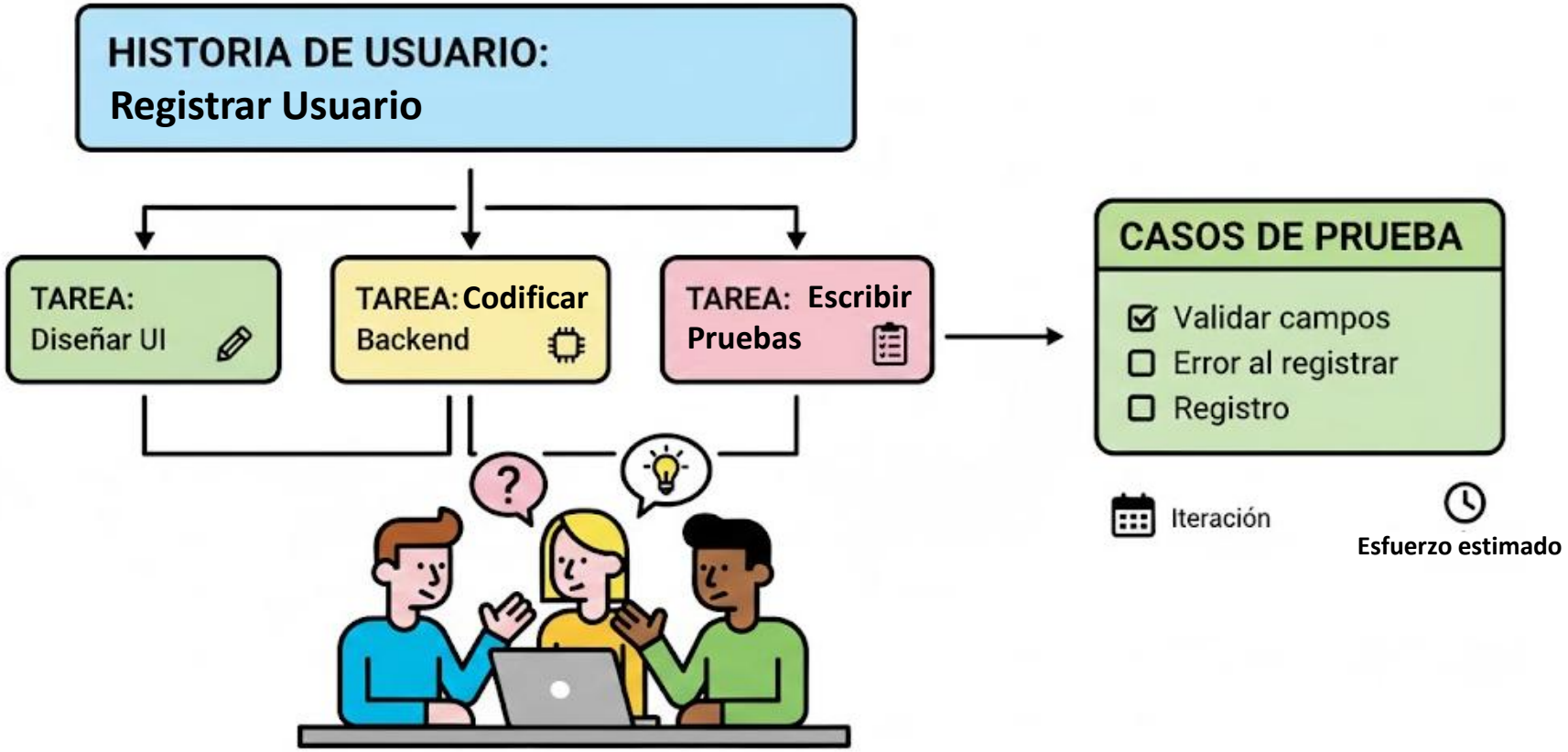
idat

Consejos de buenas prácticas:

- Siempre escribir las historias con el qué, evitar el cómo.
- No escribir una descripción exhaustiva, sólo lo justo.
- Escribir el criterio de aceptación de forma suficientemente explícita.
- Estimar todas las historias. No hacerlo puede crear falsas expectativas.
- No fiar toda la información a las tarjetas. Se puede usar documentación externa, como una wiki.
- Nunca dar una historia por finalizada antes de cumplir con todos los requerimientos.

idat

Creación de casos de prueba basados en historias de usuario



idat

Creación de casos de prueba basados en historias de usuario

1. **Entender la historia de usuario:** Analiza la historia para comprender el rol del usuario, la necesidad y el propósito de la funcionalidad. Las historias de usuario deben ser INVEST (independientes, negociables, valiosas, estimables, pequeñas y testeables) para facilitar las pruebas.

idat

Creación de casos de prueba basados en historias de usuario

2. **Identificar escenarios:**
   - **Escenario positivo (camino feliz):** Describe la secuencia de pasos que el usuario seguiría para que la funcionalidad opere correctamente.
   - **Escenarios negativos:** Explora cómo reacciona el sistema ante entradas o situaciones inesperadas (p. ej., iniciar sesión con una contraseña incorrecta).
   - **Escenarios de regresión:** Verifican que los cambios no afecten otras partes de la aplicación o la funcionalidad general de los módulos.
   - **Análisis de límites:** Prueba los puntos de entrada mínimo y máximo para asegurar que el sistema se comporta correctamente en los bordes.
   - **Escenarios de integración:** Si se modificó un módulo, se debe verificar que sigue funcionando correctamente con los demás módulos con los que interactúa.

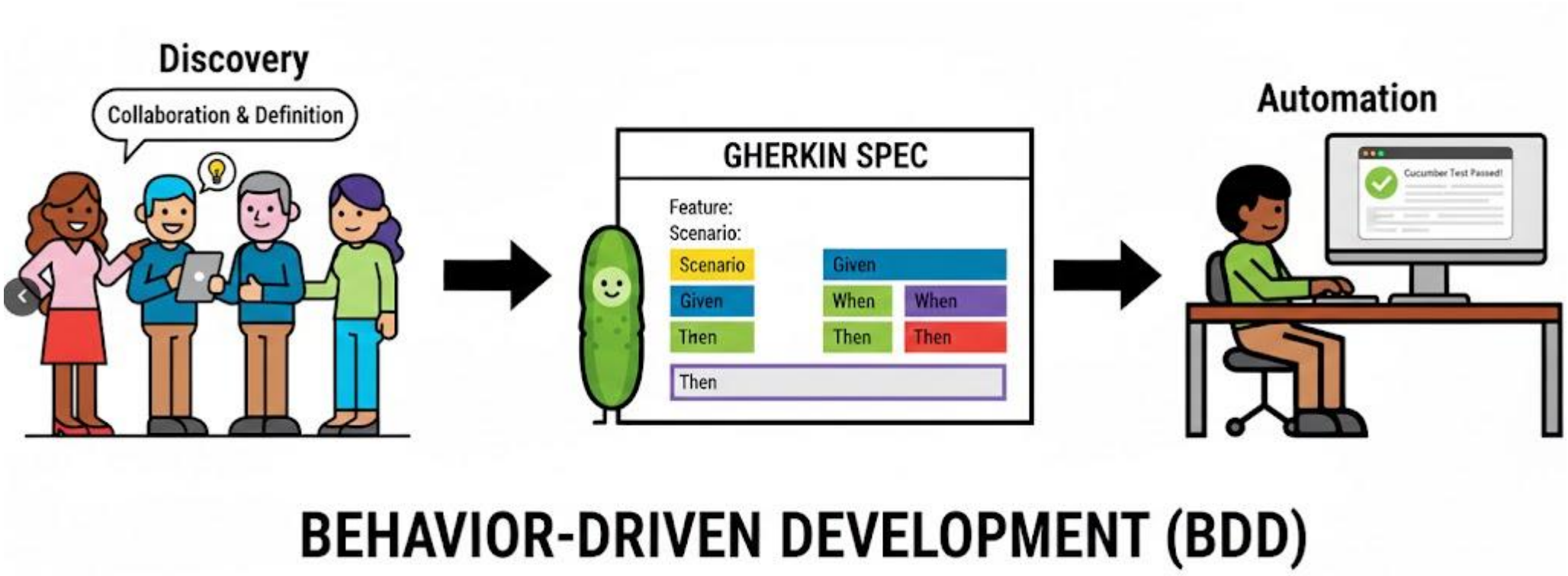
idat

Creación de casos de prueba basados en historias de usuario

3. **Escribir los pasos de prueba:**
   - Para cada escenario, crea una secuencia de pasos lógicos y claros.
   - Describe la acción que el usuario realiza en cada paso.
   - Define claramente el resultado esperado después de cada acción. Un resultado esperado describe cómo debería comportarse la aplicación.
4. **Detallar los pasos de prueba:**
   - Para cada paso de prueba, puedes añadir datos adjuntos si es necesario.
   - Utiliza una herramienta de gestión de pruebas para vincular automáticamente los casos de prueba a las historias de usuario correspondientes y mantener la trazabilidad.

idat

Behavior-Driven Development (Desarrollo Guiado por el Comportamiento)



idat

¿Qué es BDD?

BDD es Behavior Driven Development, o lo que es lo mismo en español, **desarrollo guiado por comportamiento**. Es un proceso de software ágil que busca la colaboración y entendimiento entre desarrolladores, gestores de proyecto y equipo de negocio. Es decir, es el camino para tomar antes de la fase de testing de un proyecto.

De hecho, tiene relación directa con el testing, pues el **BDD surge directamente del TDD o desarrollo guiado por pruebas**. Pero a diferencia del TDD, el BDD define las **pruebas centradas en el usuario y el comportamiento del sistema** y no en las funcionalidades de este. En otras palabras, el BDD describe las pruebas en un lenguaje natural que entienden todos los equipos de un proyecto, y no únicamente los programadores.

idat

¿Cuándo usar el desarrollo guiado por comportamiento?

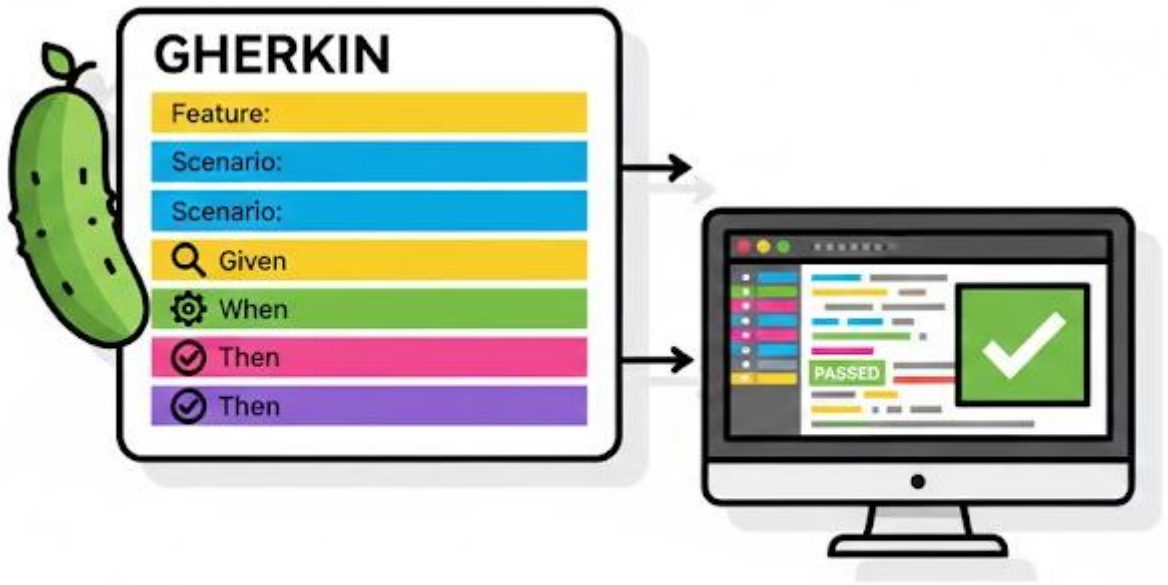
El BDD es perfecto para **comunicarse con todos los equipos durante el desarrollo de productos multifuncionales**. Un equipo de desarrolladores y analistas QA trabaja en el código mediante un lenguaje de programación específico que en muchas ocasiones el cliente o cualquier otro equipo del proyecto no es capaz de leer.

El BDD permite a todos los implicados **entender el proceso desarrollado y el contenido del código fuente**. De hecho, este desarrollo le indica al código qué tiene que hacer y lo hace en un lenguaje natural entendido por todo el mundo.

Beneficios del BDD

- **Aumenta la colaboración entre las partes involucradas.** Al tener todos un mismo lenguaje de entendimiento, todas las partes sienten que forman un único equipo y mejora la colaboración entre ellas.
- **Se prioriza el valor de negocio.** Se establecen prioridades con los clientes y el lenguaje BDD permite a los desarrolladores entender e involucrarse más con los objetivos comerciales, logrando un resultado óptimo a las necesidades del cliente.
- **Reducción de riesgo en el proyecto.** Como todas las partes implicadas saben de qué se está hablando en todo momento y cuentan con una visión global del proyecto, se corren menos riesgos en el desarrollo e implementación del proyecto.
- **Creación de metodologías ágiles de trabajo.** Al explicarse todos los requerimientos y escenarios posibles, el desarrollo posterior es mucho más sencillo, ya que se intenta comprender el comportamiento del sistema en todas sus vertientes.

Cucumber y gherkin



Cucumber

Es una herramienta de pruebas de software de código abierto escrita en Ruby. Permite escribir casos de prueba que cualquiera puede entender fácilmente, independientemente de sus conocimientos técnicos

idat

¿Qué es Gherkin?

Gherkin (Pepinillo en español), es en si un DSL, Lenguaje Específico de Dominio, esto significa que es un lenguaje que se creó específicamente para resolver un problema en particular proveyéndonos también de una técnica que nos permite solucionar dicho problema.

¿Cuál es nuestro problema? **La comunicación efectiva.**

La comunicación entre las partes de un equipo siempre va a ser un problema, más específicamente la comunicación entre lo que se denominó posteriormente “The Three Amigos” es decir, el Product Owner (PO), el desarrollador y el tester. Ahora ¿Cómo nos ayuda este lenguaje a mejorar la comunicación entre dichas partes?

idat

¿Qué es Gherkin?

Gherkin nos provee 6 palabras clave:

Feature (Característica, Funcionalidad): Funcionalidad del sistema.
Background (Antecedentes): Precondiciones del escenario.
Scenario (Escenario): Situación general.
Given (Dado): Contexto de la situación.
When (Cuando): Acción.
Then (Entonces): Resultado esperado.

idat

¿Qué es Gherkin?

Supongamos que el PO cita a los desarrolladores y a los testers, para presentar una nueva funcionalidad del sistema, la funcionalidad consiste en las posibilidades que tiene un usuario de publicar blogs en una página. Todos acordamos en utilizar Gherkin para describir los criterios de aceptación de nuestra funcionalidad para mejorar nuestra comunicación, por lo tanto luego de unos minutos de charla sobre lo presentado por el PO llegamos a un documento como el siguiente:

```
(Feature)
Característica: Publicaciones en blogs
 Solo los dueños de un blog pueden publicar en él, excepto los administradores, que pueden
publicar en todos los blogs.
  (Background)
  Antecedentes:
    Dado un administrador llamado "Pepe"
    Y un blog llamado "La vida es más feliz con Pepe" propiedad de "Pepe"
    Y un usuario llamado "Esteban"
    Y un blog llamado "Todo sobre autos" propiedad de "Esteban"

  Escenario: Esteban publica en su propio blog
    Dado que estoy logueado como Esteban
    Cuando intento publicar en "Todo sobre autos"
    Entonces debería ver "Tu artículo fue publicado exitosamente"

  Escenario: Esteban intenta publicar en el blog de otra persona
    Dado que estoy logueado como Esteban
    Cuando intento publicar en "La vida es más feliz con Pepe"
    Entonces debería ver "¡Acceso denegado! ¡Ese no es tu blog!"

  Escenario: Pepe publica en el blog de un usuario
    Dado que estoy logueado como Pepe
    Cuando intento publicar en "Todo sobre autos"
    Entonces debería ver "Tu artículo fue publicado, a Esteban no le va a gustar esto."
```

idat

¿Qué es Gherkin?

Como podemos observar, los escenarios no son más que situaciones o ejemplos que pueden darse en nuestra funcionalidad, y las palabras: Dado, Cuando, Entonces (Given, Then, When) nos ayudan a describir claramente lo que debería de suceder en cada una de estos casos.

¿Quién lo redacta? Es indiferente, la idea es que todos los participantes, los "Three Amigos" tengan una charla fluida en la cual se presenten los criterios de aceptación iniciales y a partir de ahí puedan surgir más ideas y escenarios que se puedan evaluar en ese momento o que el PO se los lleve para rever con el cliente. Por dar un ejemplo, en este caso alguno de los integrantes podría preguntarse:

> ¿Puede el administrador "Pepe" editar un artículo ya publicado por un usuario? o
> ¿Puede el administrador "Pepe" publicar en un blog de otro administrador?

idat

¿Qué es Gherkin?

La idea es intentar cubrir todos los casos que se nos vengan a la mente para que todos estemos de acuerdo en cómo debe comportarse nuestro sistema. ¿De qué nos ayuda tenerlo escrito de esta forma? Es un lenguaje natural y estructurado y fácil de entender por una persona técnica así como no técnica, es un estándar. Cualquier cliente podría entender los requerimientos sin ser un experto, digamos que hace que las especificaciones funcionales sean digeribles. Y la frutilla del postre, ayuda a que luego dichas frases puedan ser utilizadas como la base de automatizaciones, pero llegaremos allí luego.

idat

¿Qué origen tiene Gherkin?

Había una vez un desarollador con más de 20 años de experiencia en el rubro, este desarrollador lleva el nombre de Daniel Terhost-North. Dan formaba parte de un equipo el cual desarrollaba bajo la modalidad de TDD (Test Driver Development) (Desarrollo Guiado por Pruebas).

TDD en pocas palabras busca basar nuestro desarrollo en pruebas que escribimos antes de escribir el código de nuestro sistema, por más confuso que suene, tiene sentido, supongamos que estamos diseñando una aplicación que tiene como funcionalidad la suma de dos números enteros:

```
// Test de mi Sistema
Operaciones_Suma_SumaDeEnterosExitosa:
    resultado = sumar(2,2)
    assert resultado == 4

// Funcionalidad de mi sistema
Operaciones_Suma_SumaDeEnterosExitosanumero_1,numero_2):
    return numero_1+numero_2
```

idat

¿Qué origen tiene Gherkin?

El problema es que seguían las mismas confusiones una y otra vez bajo este modelo: Por dónde empezar, qué testear y qué no testear, cuánto testear en un mismo test, cómo nombrar dichos tests, cómo entender por qué un test falla.

(Cabe resaltar que TDD hace referencia a la etapa de desarrollo, es decir, **tests escritos por los desarrolladores**)

Todo esto en resumidas cuentas lo impulsó a dar nacimiento a una nueva metodología "BDD".

¿Qué origen tiene Gherkin?

Si en TDD los desarrolladores creaban scripts de pruebas que su código debía superar para así cerciorarse de que su trabajo cumplía con la consigna, en BDD se creaban scripts de pruebas que el código debía superar de igual forma pero bajo la perspectiva del comportamiento del usuario. Si bien puede sonar algo confuso, lo vamos a entender cuando conectemos **BDD** con **Gherkin**.

Cabe aclarar que **BDD** no tiene relación en su nacimiento con **Gherkin**, si no que años después de nacer **BDD**, Daniel North junto con Chris Matts y Liz Keogh toman un template de requerimientos y lo convierten en un documento que hace uso de palabras claves en inglés así como vimos anteriormente, llevándonos así al nacimiento del primer template de **Gherkin**.

idat

¿Qué relación tiene Gherkin con BDD?

Como vimos anteriormente, **Gherkin** es sólo una forma de escribir requerimientos de un sistema y **BDD** es una metodología de desarrollo que consiste en escribir los tests basados en el comportamiento del usuario antes siquiera de escribir el código del sistema.

¿Qué tiene que ver un documento funcional (**Gherkin**) con los tests que programen los desarrolladores para luego verificar su código (**BDD**)?

Allí es donde entra **Cucumber**.

¿Qué es Cucumber?

Cucumber (o Pepino en español) sí, le pusieron así para hacer el juego entre **Gherkin** y **Cucumber**, es una herramienta creada por Aslak Hellesøy que ayuda a los desarrolladores a programar bajo la metodología **BDD** en distintos lenguajes, haciendo uso de tests redactados en **Gherkin**.

Como vimos en nuestro ejemplo sobre la funcionalidad de blogs en **Gherkin**, podemos decir que al final del día estamos describiendo el comportamiento del usuario en ciertos escenarios, es decir, que cada uno de esos ejemplos sería un posible camino o situación experimentada por un usuario, teniendo entonces estrecha relación con el "comportamiento" mencionado por **BDD** (Desarrollo Guiado por Comportamiento). Pero ¿Cómo conectan los desarrolladores ese comportamiento descrito en **Gherkin** con tests que les ayuden a probar el sistema?

¿Qué es Cucumber?

Con **Cucumber**!

Veamos un ejemplo con pseudocódigo para que se hagan una idea:

```
Escenario: Pepe publica en el blog de un usuario
  Dado que estoy logueado como Pepe
  Cuando intento publicar en "Todo sobre autos"
  Entonces debería ver "Tu artículo fue publicado, a Esteban no le va a gustar esto."

SCRIPT DE CUCUMBER (en pseudocódigo)
STEP
  @Dado que estoy logueado como Pepe
      SUB-STEPS:
        1.  Ir a  <Home Page>
        2.  Hacer click en <botón_iniciar_sesión>
        3.  Ingresar <email_pepe> en casilla <email_login>
        4.  Ingresar <contraseña_pepe> en casilla <contraseña_login>
        5.  Hacer click en <botón_continuar>
        6.  Verificación logueo exitoso.
STEP
  @Cuando intento publicar en "Todo sobre autos"
      SUB-STEPS:
        1.  Ir a <Todo Sobre Autos>
        2.  ...
        3.  ...
STEP
  @Entonces debería ver "Tu artículo fue publicado, a Esteban no le va a gustar esto."
        1.  ...
```

idat

¿Qué es Cucumber?

Gracias a **Cucumber** todos los escenarios que redactamos en nuestro **Gherkin** ahora serán plantillas para disparar tests automatizados, convirtiendo cada oración de nuestro escenario en algo llamado “Step” o paso y asignándole a este paso una secuencia de acciones/funciones llamadas “Sub Steps” o sub pasos que por detrás ejecutan código.

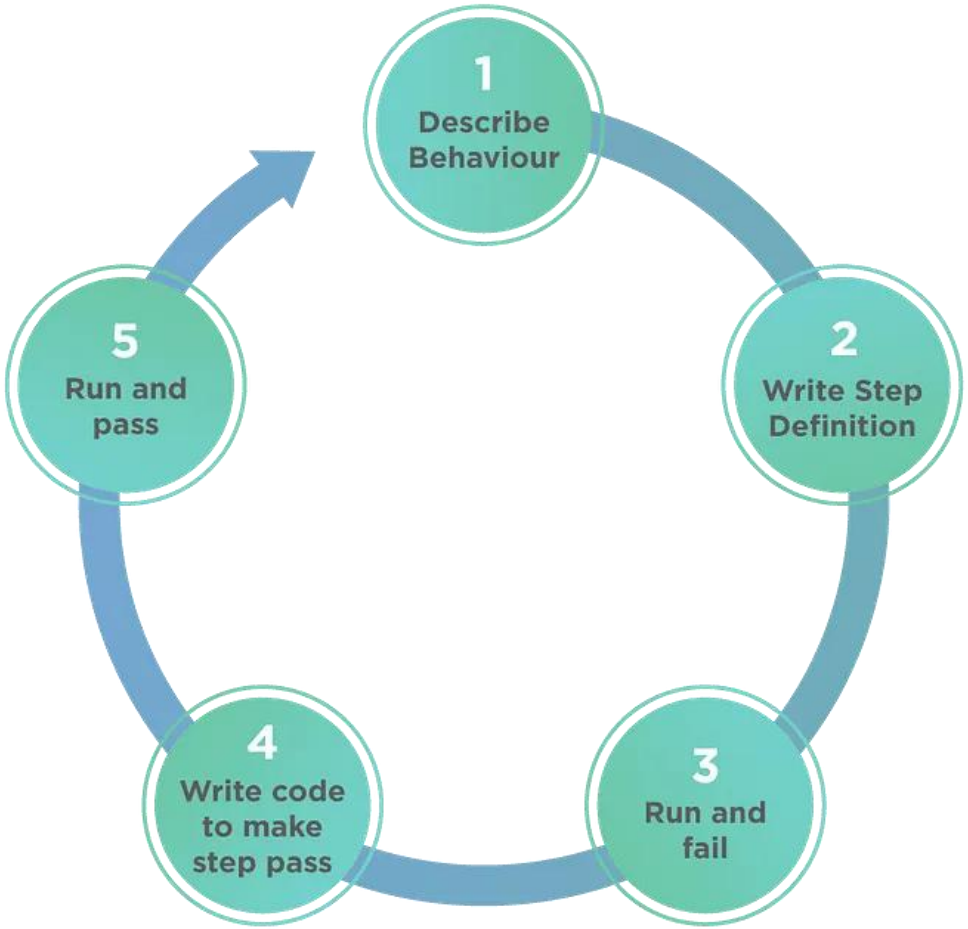
Ahora los desarrolladores podrán convertir los escenarios de **Gherkin** en disparadores de tests que corroboren que cada oración (Step) se cumpla, es decir, cuando se corra el test asociado al escenario “**Pepe publica en el blog de un usuario**” y se ejecute la oración o step “**Dado que estoy logueado como Pepe**” se espera que el script corra un cierto código haciendo uso del sistema que implique **loguear exitosamente a un usuario administrador**, sí el script falla, el step falla y la condición no se cumple, haciendo así que falle el escenario y por consecuencia, falla el test.

¿Qué es Cucumber?

En el caso de que todos los Steps (Dado, Cuando, Entonces) se ejecuten exitosamente, entonces el desarrollador habría logrado cumplir con el requerimiento expresado en dicho escenario.

idat

Uniendo Gherkin + BDD + Cucumber



idat

Uniendo Gherkin + BDD + Cucumber

1. **Se describe el comportamiento:** Etapa en donde se realizaría la reunión de los tres amigos para discutir la funcionalidad del sistema ejemplificando con escenarios que resulten en la redacción de un documento redactado en **Gherkin**. (Tarea llevada a cabo por el PO, desarrolladores y testers)
2. **Se escriben las definiciones de los Steps:** es decir el contenido de scripts que va a asegurar que nuestro sistema esté haciendo lo que debería hacer teniendo en cuenta el comportamiento del usuario (Tarea llevada a cabo por desarrolladores)
3. **Estos tests automatizados son corridos por los desarrolladores** y van a fallar porque todavía no se escribió el código que logre que estas pruebas pasen.
4. **Los desarrolladores escriben el código hasta lograr crear la nueva funcionalidad** que cumple con los requisitos expresados en Gherkin.
5. **Se corren los tests nuevamente y ahora los tests que fallaban al comienzo se ejecutan exitosamente** al haberse logrado la funcionalidad deseada.

idat

¿Dudas?

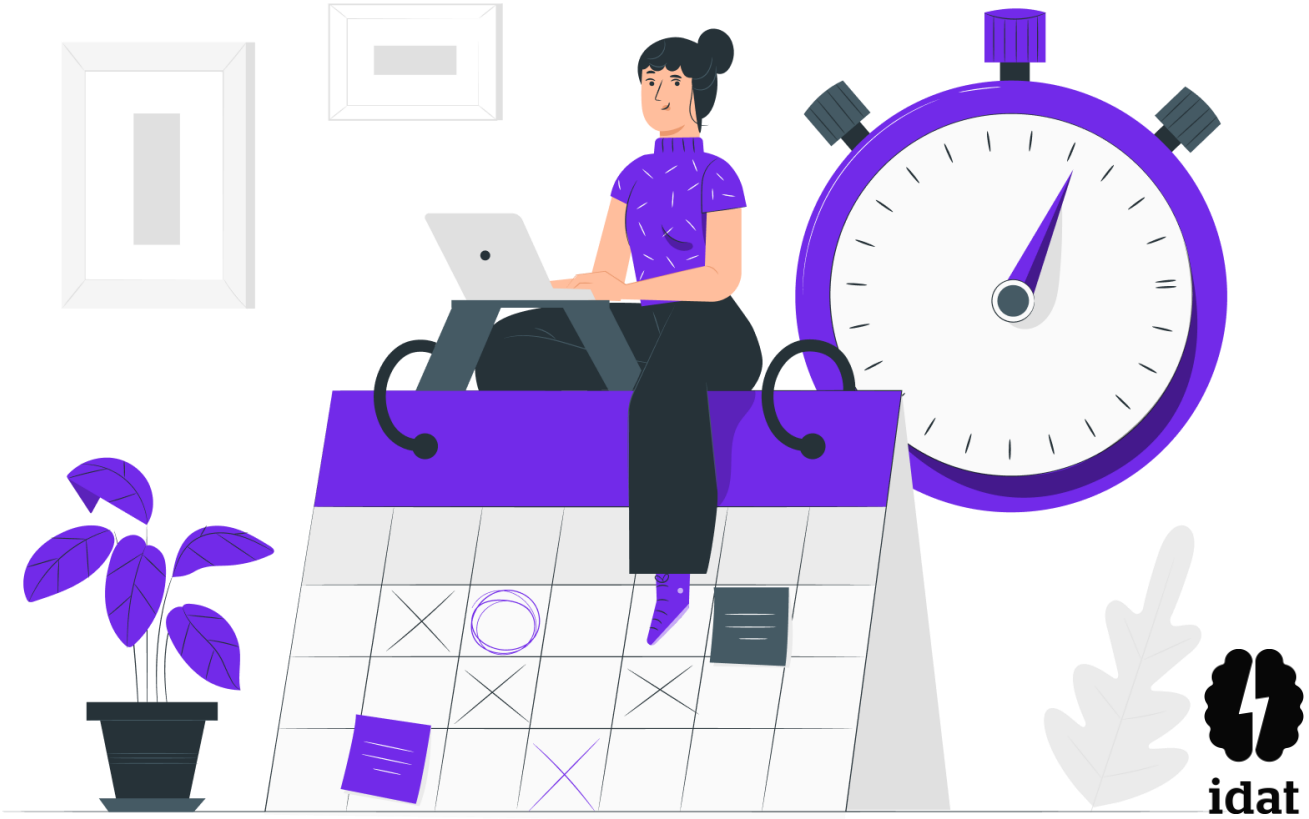
¿Miedos?

¿Temores?

idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS